# EventPilot — An AI Event Planning Agent

### EECS 3311 Software Design · Fall 2026 · Course Project — Stage 1 Design Report

| | |
|---|---|
| **Student** | Mohammed Faraz Kabbo |
| **Student ID** | *(fill in)* |
| **Repository** | `https://github.com/<your-username>/EECS3311-EventPilot` (used for all three stages) |
| **Date** | October 2026 |

**Contents**


1. Project Overview
2. Detailed Feature Specifications
3. UML Class Diagram
4. Design Pattern Explanations
5. Use-Case Diagram
6. Detailed Use-Case Descriptions
7. Sequence Diagrams
8. Feature-to-Design Traceability Table
9. Feature Implementation Explanations


All diagrams are drawn in PlantUML. The sources are in `docs/stage1/diagrams/src/`, and rendered PNG/SVG files are in `docs/stage1/diagrams/png` and `svg`. The same class and method names are used in every section.

---

## 1. Project Overview

### 1.1 Problem and Motivation

Planning an event such as a 120-person meetup, a student conference or a company workshop means juggling many constraints that affect each other. Sessions must not overlap in the same room. Speakers are only free at certain times. The venue must fit the crowd and the budget. Catering must match confirmed RSVPs and dietary needs. Dozens of tasks have deadlines that depend on each other. And the plan keeps changing ("move the keynote to 2 pm").

Small organizers (student clubs, community groups, small businesses) usually do this with spreadsheets, e-mail threads and a generic chatbot. Spreadsheets can do the arithmetic but cannot suggest anything. A chatbot can suggest, but it does not know the real state of the event, invents numbers, and cannot safely change anything. The result is double-booked rooms, forgotten dietary restrictions and budget overruns.

**EventPilot** combines a **deterministic event-management core** with an **AI planning agent**. The core holds the data model, checks conflicts, does the budget arithmetic, enforces the task rules and stores everything. The agent reasons over the real event, calls the core's functions as **tools**, and proposes concrete changes. The organizer approves each proposal before it is applied, and can undo it afterwards.

### 1.2 Target Users

| User | Need |
|---|---|
| **Event organizer** (primary): student-club executives, community coordinators, small-business staff | Turn a rough idea into a complete, consistent plan quickly, and keep it correct as things change |
| **Co-organizer** | See and update the tasks assigned to them |
| **Attendee** | Get quick, correct answers about the event (schedule, food, location) |
| **Power user / TA** | Use every major function from a terminal (CLI) |

### 1.3 What the Agent Does

The agent is a **tool-using planning agent with memory, retrieval and a human-approval step**. It can:

- **Understand:** extract a structured event from a plain-language description, and ask follow-up questions when information is missing.

- **Plan:** create an agenda, schedule speakers, split the budget, design catering and break the event into tasks.
- **Use tools:** search venues, compute budgets and catering portions, check speaker availability, query attendees and run the conflict checker (6 tools).
- **Decide:** rank venues with trade-offs, choose how to fix conflicts, and decide when an attendee question cannot be answered and must go to the organizer.
- **Act safely:** every change it suggests is a list of `EventCommand` objects in a `PlanProposal`. Nothing changes until the organizer approves, and every applied change can be undone.
- **Remember:** conversation memory resolves words like "it" and "the keynote"; preference memory keeps habits such as "prefers 15-minute breaks".
- **Retrieve:** answer attendee questions only from the event's own information, citing the source (RAG).

### 1.4 Why an AI Agent Fits This Problem

| Property of the problem | What a normal app cannot do | What the agent adds |
|---|---|---|
| Requests arrive in natural language | Forms require every field up front | Extraction plus clarifying questions (F01, F12) |
| Planning is open-ended (agenda shape, menu, budget split) | Fixed rules cannot cover every reasonable plan | The LLM generates plans that fit the constraints |
| Plans must obey hard constraints | A chatbot alone invents numbers | The agent calls deterministic tools, and guardrails check its output |
| Work is multi-step (search → check budget → rank → explain) | One prompt cannot gather information step by step | The orchestrator runs a loop of up to 8 tool-using steps |
| The organizer must stay in control | Fully automatic changes are risky | Proposals need approval, and changes can be undone |

**Design principle:** *the LLM reasons and proposes; deterministic code computes, checks and applies.*

### 1.5 AI / LLM Models

| Role | Model | Used for |
|---|---|---|
| Main planning model | **Claude Sonnet 5.5** (`claude-sonnet-5-5`) via the Anthropic Messages API | Planning with tools: agenda, scheduling, conflicts, budget, catering, tasks, plan changes, event extraction |
| Lightweight model | **Claude Haiku 4.5** (`claude-haiku-4-5-20251001`) | Fast, cheap tasks: FAQ answers, invitation drafts |
| Offline option | **Llama 3.1 8B** via **Ollama** (local) | Development without API cost, offline demos |
| Test double | `MockLLMProvider` (scripted replies) | Repeatable automated tests |

The model is chosen in a configuration file (`AppConfig`) and created by `LLMProviderFactory`. Switching models therefore needs no code change (Strategy pattern).

### 1.6 How the AI Interacts with the Rest of the System

The AI never touches the GUI or the database directly. Every AI feature follows the same path:

1. The **GUI controller or CLI command** calls one method on `EventPlannerFacade`.
2. The facade runs the matching **workflow** (`PlanningWorkflow` subclass). The workflow collects context (event data, memory, retrieved documents) and builds the prompt.
3. `AgentOrchestrator` sends the prompt to the LLM through `LLMClient`. When the LLM asks for a tool, `ToolRegistry` checks the arguments and runs the tool, which is ordinary Java code. The result goes back to the LLM. This repeats for at most 8 steps.
4. The workflow **parses** the LLM's JSON answer into objects. If the JSON is invalid it asks the LLM to fix it, up to 2 times. It then **validates** the result with `GuardrailValidator` (capacity, budget, time window, state rules).
5. The facade returns a `PlanProposal` that holds a list of `EventCommand`s. The organizer **reviews** it in a dialog (GUI) or a y/N prompt (CLI).
6. On approval, `CommandHistory` executes the commands, `EventRepository` saves the event, and `EventBus` notifies the views, the alert service and the FAQ index.

## 1.7 Overall Architecture

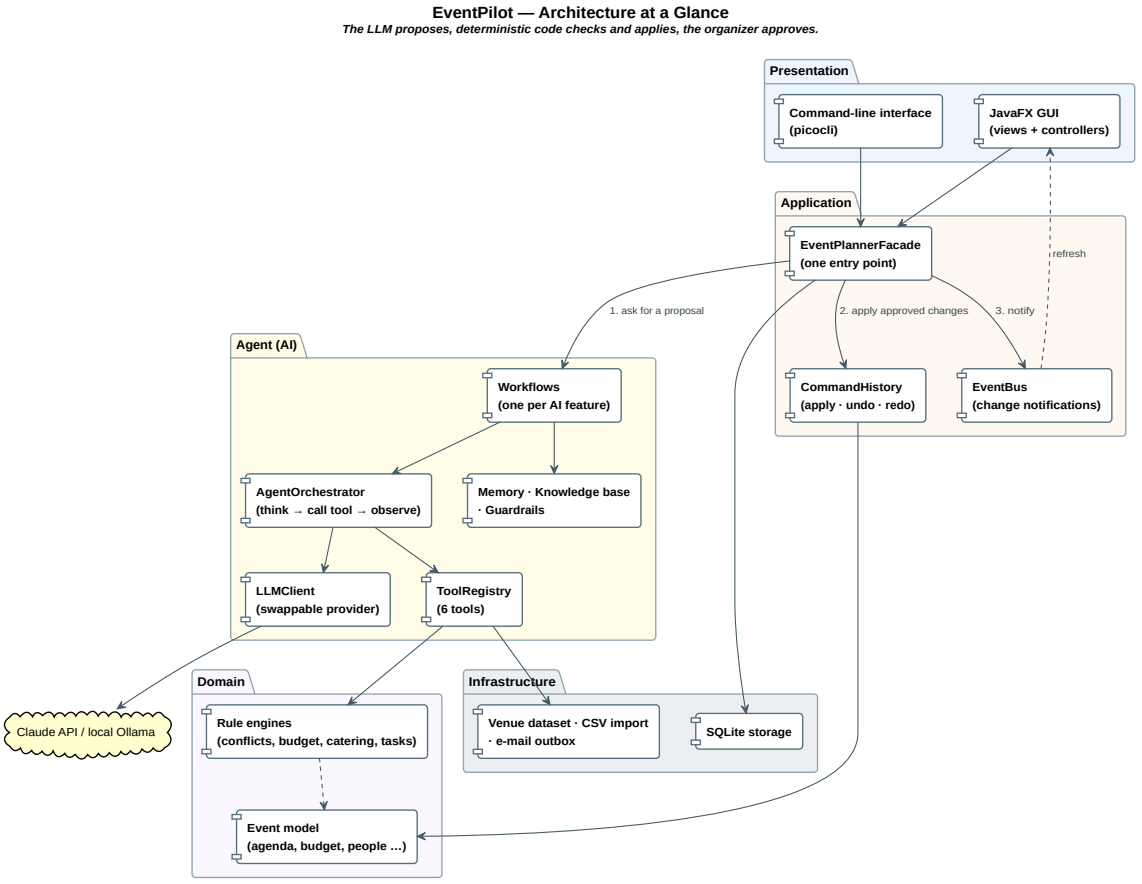



*Figure 1 — Layered architecture.*

| Layer | Responsibility | Main classes |
|---|---|---|
| Presentation | JavaFX GUI (MVC views and controllers) and picocli CLI | `BaseView` + 6 views, `ChatPanel`, `ProposalReviewDialog`, `BaseController` + 6 controllers, `BaseCliCommand` + 10 commands |
| Application | One entry point; apply, undo and redo changes; notify about changes | `EventPlannerFacade`, `CommandHistory`, `EventCommand`s, `EventBus`, `AlertService`, `KnowledgeBaseIndexer` |
| Agent | Workflows, agent loop, LLM access, tools, memory, retrieval, guardrails | `PlanningWorkflow` + 11 subclasses, `AgentOrchestrator`, `LLMClient`, `LLMProvider`, `ToolRegistry`, 6 tools, `MemoryManager`, `KnowledgeBase`, `GuardrailValidator` |
| Domain | The event model, lifecycle states and rule engines | `Event`, `EventState` + 5 states, `Agenda`, `Budget`, `Attendee`, `ConflictDetector`, `BudgetTracker`, `CateringCalculator`, `TaskScheduler` |
| Infrastructure | Storage, venue data, CSV import, e-mail outbox, configuration | `SqliteEventRepository`, `JsonVenueCatalogAdapter`, `AttendeeCsvImporter`, `OutboxFileGateway`, `AppConfig` |

**Technology:** Java 21, Maven, JavaFX, picocli, Jackson, SQLite (JDBC), Anthropic API / Ollama, JUnit 5 + Mockito (Stage 3A), KUMA (Stage 3B).

**Requirement check:** GUI ✔ · CLI ✔ · 12 features (≥ 10) ✔ · 7 design patterns (≥ 5) ✔ · LLM integrated ✔ · agent behavior: planning, tool use, retrieval, memory, decision making, multi-step execution ✔

---

## 2. Detailed Feature Specifications

| ID | Feature | Type | GUI | CLI |
|---|---|---|---|---|
| F01 | Create event from a description | AI + deterministic validation | Chat panel | `event create "<text>"` |
| F02 | Attendee & RSVP management | Deterministic | Attendees tab | `attendee add / import / rsvp` |
| F03 | Venue recommendation | Hybrid (tool use) | Venue tab | `venue recommend` |
| F04 | Agenda generation | Hybrid | Agenda tab | `agenda generate` |
| F05 | Speaker scheduling | Hybrid (tool use) | Agenda tab | `agenda schedule` |
| F06 | Conflict detection & resolution | Hybrid | Agenda tab | `agenda conflicts / resolve` |
| F07 | Budget planning & expense tracking | Hybrid | Budget tab | `budget generate / expense / status` |
| F08 | Dietary-aware catering plan | Hybrid | Attendees tab ▸ Catering | `catering plan` |
| F09 | Task plan & tracking | Hybrid | Tasks board | `task generate / list / status` |
| F10 | Invitation drafting & sending | AI + deterministic delivery | Attendees tab ▸ Invitations | `invite draft / send` |
| F11 | Attendee FAQ agent (RAG) | AI (retrieval-based) | Chat panel, FAQ mode | `ask "<question>"` |
| F12 | Plain-language plan changes with undo/redo | Hybrid | Chat panel, Agenda tab | `agenda modify "<text>"`, `agenda undo / redo` |

"Hybrid" means that the AI proposes and deterministic code computes and validates.

### F01 — Create Event from a Description

| | |
|---|---|
| **Description** | The organizer describes the event in plain language. The agent extracts the name, type, date/time, expected attendance, budget and city, and asks follow-up questions if something required is missing. Once complete, the event is created in the *Draft* state. |
| **User Interaction** | Types in the **Chat panel** (Dashboard ▸ New event ▸ Describe it) and answers follow-up questions in the same chat. |
| **Input** | Free-text description; answers to follow-up questions; saved preferences (e.g. default city). |
| **Output** | A saved `Event` (state = Draft) and a confirmation message. |
| **AI Involvement** | **AI** extraction with **deterministic** validation (`GuardrailValidator.validateDraft`, `EventDraft.missingFields`). |
| **Workflow** | 1. `AgentChatController.submitMessage()` → `EventPlannerFacade.createEventFromDescription()`. 2. `EventExtractionWorkflow` builds the prompt and runs the agent. 3. The reply is parsed into an `EventDraft` and validated. 4. If fields are missing, questions are returned and answers come back through `answerClarification()`. 5. Otherwise `toEvent()` creates the event, it is saved, and the Dashboard refreshes. |
| **Errors / Alternatives** | LLM unreachable after 3 retries → "use the manual New Event form". Invalid JSON → the LLM is asked to fix it (max 2). Contradictions (end before start) → asked as a question, never guessed. Request is not about an event → politely declined. |

### F02 — Attendee & RSVP Management

| | |
|---|---|
| **Description** | Add, edit and remove attendees; import a CSV with row-by-row validation and duplicate detection; record role, dietary restrictions and accessibility needs; track RSVP status. |
| **User Interaction** | **Attendees tab**: table, *Add* form, *Import CSV* button, RSVP drop-down per row, RSVP pie chart. |
| **Input** | Form fields, or a CSV file (`name,email,role,dietary`); RSVP changes. |
| **Output** | Updated attendee list; `ImportReport` (imported / rejected rows with line numbers / duplicates). |
| **AI Involvement** | **Deterministic.** F08, F10 and F11 use this data. |
| **Workflow** | `AttendeeController.onImportCsv()` → `importAttendees()` → `AttendeeCsvImporter.parse()` → valid attendees added → saved → `EventBus.publish(ATTENDEES_CHANGED)` → views refresh and the FAQ index updates. |
| **Errors / Alternatives** | Missing file or wrong header → error that shows the expected header. Bad rows → rejected individually while the rest import. Duplicate e-mails → skipped and reported. Event Completed/Cancelled → editing refused. |

## F03 — Venue Recommendation

| | |
|---|---|
| **Description** | The agent searches the venue dataset with the `venue_search` tool (capacity, city, accessibility, price), checks the venue budget with `budget_calculator`, and returns the top 3 venues with a score, a reason and trade-offs. The organizer picks one. |
| **User Interaction** | **Venue tab ▸ Recommend venues**; three venue cards; the review dialog asks the organizer to choose one. |
| **Input** | The event (attendance, city, date, venue budget); optional extra needs ("needs a stage"). |
| **Output** | `PlanProposal` with up to 3 `SetVenueCommand`s; after approval, the selected `Venue`. |
| **AI Involvement** | **Hybrid**: searching is deterministic (inside the tool), ranking and explaining is AI, and checks are deterministic. |
| **Workflow** | See SD02. `recommendVenues()` → `Event.ensureEditable()` → `VenueRecommendationWorkflow` → agent loop with `VenueSearchTool` → `JsonVenueCatalogAdapter.search()` → parse → validate → review → `applyProposal()`. |
| **Errors / Alternatives** | Invalid tool arguments → rejected by `ToolRegistry` and sent back to the LLM. No match → the agent may relax only price or district, and says so. A venue not returned by the tool (invented) → blocked by validation. Step limit reached → "refine your criteria". |

## F04 — Agenda Generation

| | |
|---|---|
| **Description** | Creates a full agenda (sessions, keynote, breaks, meals) inside the event hours, using the organizer's preferences (number of tracks, session length, lunch time). |
| **User Interaction** | **Agenda tab ▸ Generate agenda** → small preferences dialog → review dialog → timeline view. |
| **Input** | Event hours, rooms, speakers and topics, `AgendaPreferences`, saved preferences. |
| **Output** | `PlanProposal` of `AddAgendaItemCommand`s; after approval, the updated `Agenda`. |
| **AI Involvement** | **Hybrid**: the AI designs the structure; the `conflict_check` tool and `GuardrailValidator` ensure correctness. |
| **Workflow** | See SD03. `onGenerateAgenda()` → `generateAgenda()` → `AgendaGenerationWorkflow` → parse with `ProposalParser`/`CommandMapper` → validate → review → `CommandHistory.execute(MacroCommand)` → save → publish. |
| **Errors / Alternatives** | Items outside event hours or overlapping → warnings (HIGH severity blocks approval). Agenda already exists → choose *replace* or *extend*. Organizer rejects → nothing changes. |

## F05 — Speaker Scheduling

| | |
|---|---|
| **Description** | Assigns speakers to sessions and times so that each speaker is only scheduled when available and never double-booked. |
| **User Interaction** | **Agenda tab ▸ Speakers panel** (add speakers and availability) ▸ *Auto-schedule speakers*. |
| **Input** | Speakers with availability windows, sessions, rooms. |
| **Output** | `PlanProposal` of `AssignSpeakerCommand`s and `MoveSessionCommand`s. |
| **AI Involvement** | **Hybrid**: the agent tries assignments and checks them with `speaker_availability` and `conflict_check` until no serious conflict remains. |
| **Workflow** | See SD04. `onScheduleSpeakers()` → `scheduleSpeakers()` → `SpeakerSchedulingWorkflow` → tool loop → proposal → review & apply. |
| **Errors / Alternatives** | No speakers or sessions → error before any AI call. Impossible constraints → proposal for the speakers that fit, with an explanation for the others. |

## F06 — Conflict Detection & Resolution

| | |
|---|---|
| **Description** | `ConflictDetector` (plain code) finds room overlaps, speaker double-bookings, speakers outside their availability, rooms that are too small, and items outside event hours. On request, the agent proposes a fix, which is tested on a copy of the event so it cannot create new conflicts. |
| **User Interaction** | **Agenda tab ▸ Check conflicts** (conflicts shown in red) ▸ *Ask agent to fix*. Conflicts also appear automatically as Dashboard alerts. |
| **Input** | The current event. |
| **Output** | `List<Conflict>` (type, severity, items); optionally a `PlanProposal` that fixes them. |
| **AI Involvement** | **Hybrid**: detection is deterministic; the fix is proposed by AI and checked by code. |
| **Workflow** | See SD04. `detectConflicts()` → `ConflictDetector.detect()`. `resolveConflicts()` → `ConflictResolutionWorkflow` → its `validate()` applies the fix to `event.deepCopy()` and runs `detect()` again → review & apply. |
| **Errors / Alternatives** | No conflicts → "No conflicts found". Fix creates new conflicts → blocked and retried once; if still failing, the remaining conflicts are listed for manual fixing. |

## F07 — Budget Planning & Expense Tracking

| | |
|---|---|
| **Description** | The agent proposes how to split the total budget across categories (venue, catering, AV, decor, marketing, contingency) and explains why. `BudgetTracker` does all the math. The organizer then records real expenses, sees planned vs. spent, and gets alerts. |
| **User Interaction** | **Budget tab**: *Generate budget plan*, budget table, bar chart, *Add expense* form; alerts show on the Dashboard. |
| **Input** | Budget limit, event type and size; expenses (category, amount, date, description). |
| **Output** | `PlanProposal` of `SetBudgetLineCommand`s; `BudgetStatus`; alerts. |
| **AI Involvement** | **Hybrid**: the AI chooses the split; the arithmetic and alerts are deterministic. |
| **Workflow** | See SD05. Plan: `generateBudget()` → `BudgetPlanningWorkflow` → `BudgetCalculatorTool` → `BudgetTracker.allocate()` → validate (total ≤ limit) → review & apply. Expense: `onRecordExpense()` → `recordExpense()` → `BudgetTracker.recordExpense()` → save → publish → `AlertService` checks status. |
| **Errors / Alternatives** | No limit set → asked to set one. Negative amount or missing category → form error. Plan over the limit → blocked. Category over plan or ≥ 90 % spent → alert. |

## F08 — Dietary-Aware Catering Plan

| | |
|---|---|
| **Description** | Code computes the confirmed headcount and dietary counts from RSVPs. The agent then designs meals that line up with agenda breaks, menus that cover every dietary need, portions (from the `catering_calculator` tool) and an estimated cost. |
| **User Interaction** | **Attendees tab ▸ Catering**: dietary summary, *Plan catering* button, meal cards. |
| **Input** | Confirmed attendees and their diets, meal breaks, catering budget line. |
| **Output** | `PlanProposal` with a `SetCateringPlanCommand` → `CateringPlan`. |
| **AI Involvement** | **Hybrid**: facts are computed by code, the menu by AI, and coverage and cost are checked by code. |
| **Workflow** | See SD06. `planCatering()` → `CateringPlanningWorkflow` uses `CateringCalculator.computeHeadcount()` and `summarizeDiets()` → agent loop → validate (every diet covered, cost within budget) → review & apply. |
| **Errors / Alternatives** | No RSVPs yet → uses expected attendance with a warning. A diet not covered → blocked and retried once. Over budget → warning with the amount. |

## F09 — Task Plan & Tracking

| | |
|---|---|
| **Description** | The agent breaks the event into tasks with owners, deadlines and dependencies, planning backwards from the event date. `TaskScheduler` checks for circular dependencies and orders the tasks. Co-organizers update task status on a board, and overdue tasks raise alerts. |
| **User Interaction** | **Tasks board** (To do / In progress / Done) with drag-and-drop and *Generate task plan*. |
| **Input** | Event date, type, what is already done, team member names. |
| **Output** | `PlanProposal` of `AddTaskCommand`s; task status changes; overdue alerts. |
| **AI Involvement** | **Hybrid**: the AI breaks down the work; dependency, ordering and overdue rules are deterministic. |
| **Workflow** | See SD07. `generateTasks()` → `TaskDecompositionWorkflow` → `TaskScheduler.detectCycles()` / `orderByDependencies()` → review & apply. `updateTaskStatus()` checks dependencies. `AlertService.checkOverdueTasks()` runs at start-up and every morning. |
| **Errors / Alternatives** | Circular dependency or deadline after the event → blocked and retried once. Marking a task done while a dependency is open → refused with "blocked by ...". |

## F10 — Invitation Drafting & Sending

| | |
|---|---|
| **Description** | Drafts invitations in a chosen tone (formal / friendly / playful), personalised per attendee group (speakers, VIPs, guests). Each draft is checked to contain the real date, time and venue, can be edited, and is then sent to the e-mail outbox. Allowed only when the event is in *Planning* or *Confirmed*. |
| **User Interaction** | **Attendees tab ▸ Invitations**: tone selector, *Draft*, editable previews, *Send*, delivery report. |
| **Input** | Event details, attendees and roles, tone, organizer edits. |
| **Output** | `List<Invitation>`; `DeliveryReport` (sent / failed). |
| **AI Involvement** | **AI** writing with **deterministic** fact checks and delivery. |
| **Workflow** | See SD08. `draftInvitations()` → `Event.ensureCanInvite()` (decided by the current `EventState`) → `InvitationDraftingWorkflow` → drafts shown → `sendInvitations()` → `MessageGateway.send()` for each attendee. |

| | |
|---|---|
| **Errors / Alternatives** | Wrong state → refused with guidance. Draft missing the date or venue → regenerated. A single failed delivery → recorded, and the rest are still sent. |

## F11 — Attendee FAQ Agent (RAG)

| | |
|---|---|
| **Description** | Attendees ask questions in plain language. The system retrieves the most relevant pieces of the event's information (agenda, venue, catering, organizer notes), and the agent answers **only** from them, naming its sources. If nothing relevant is found, the question goes to the organizer instead of being guessed. |
| **User Interaction** | **Chat panel ▸ FAQ mode** (can run on a shared screen). Answers show their sources. |
| **Input** | Question, event. |
| **Output** | `FaqAnswer` (text, sources, escalated flag). |
| **AI Involvement** | **AI** with deterministic retrieval (`KnowledgeBase.search`) and a source check. |
| **Workflow** | See SD09. `onAskFaq()` → `askFaq()` → `FaqAnswerWorkflow.gatherContext()` → `KnowledgeBase.search()` → agent answers from sources → `validate()` checks the cited sources were actually retrieved. |
| **Errors / Alternatives** | Nothing relevant → "I've forwarded this to the organizer" plus an alert. Answer cites an unknown source → replaced with a safe reply. Off-topic question or prompt-injection attempt → declined. |

## F12 — Plain-Language Plan Changes with Undo/Redo

| | |
|---|---|
| **Description** | The organizer changes the plan by typing, e.g. "move the keynote to 2 pm and add a 15-minute break after it". The agent turns this into operations on real agenda items, and `CommandMapper` turns those into commands. Unclear requests trigger a question. Approved changes form one `MacroCommand` that can be undone or redone as a unit. |
| **User Interaction** | Chat panel or *Modify with AI* box on the Agenda tab; *Undo* / *Redo* buttons (Ctrl+Z / Ctrl+Y). |
| **Input** | Instruction; the current agenda with item ids; recent conversation. |
| **Output** | `PlanProposal` (or a question); updated event; undo/redo history. |
| **AI Involvement** | **Hybrid**: the AI interprets intent; mapping, checking, execution and undo are deterministic. |
| **Workflow** | See SD10. `modifyPlan()` → `PlanModificationWorkflow` → `CommandMapper.toCommand()` → review → `applyProposal()` → `CommandHistory.execute()`. `undo()` → `CommandHistory.undo()` → `MacroCommand.undo()` (steps reversed). |
| **Errors / Alternatives** | Ambiguous ("which keynote?") → question. Unknown item or unsupported operation → the LLM is asked to fix it. Nothing to undo → message. A new change after undo clears the redo history. |

---

# 3. UML Class Diagram

The class diagram has one **overview** (CD0) and **four detailed views** (CD1–CD4), so that each picture stays readable. Every class has the same name and members in every view. **Colours mark design-pattern roles** and are the same in all diagrams: orange = Facade, green = Command, blue = Observer, purple = Strategy, yellow = Template Method, pink = State, cyan = Adapter.

## 3.1 CD0 — Overview

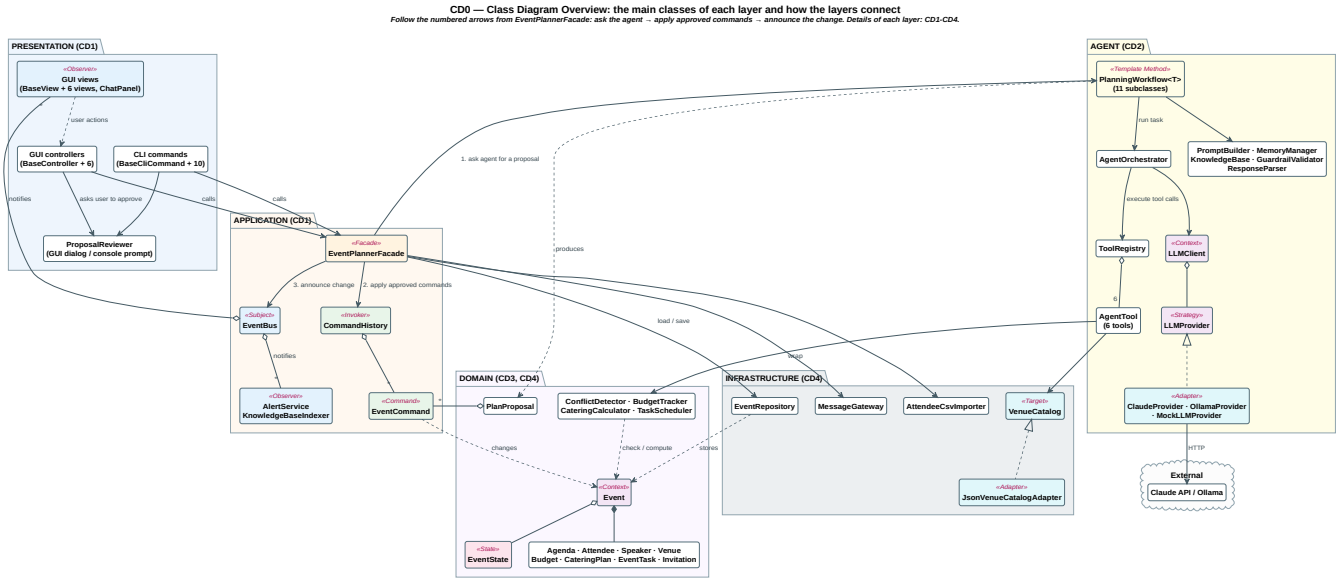



*Figure 2 — Main classes per layer. Follow the numbered arrows from `EventPlannerFacade`: (1) ask the agent for a proposal, (2) apply the approved commands, (3) announce the change.*

## 3.2 CD1 — Presentation and Application

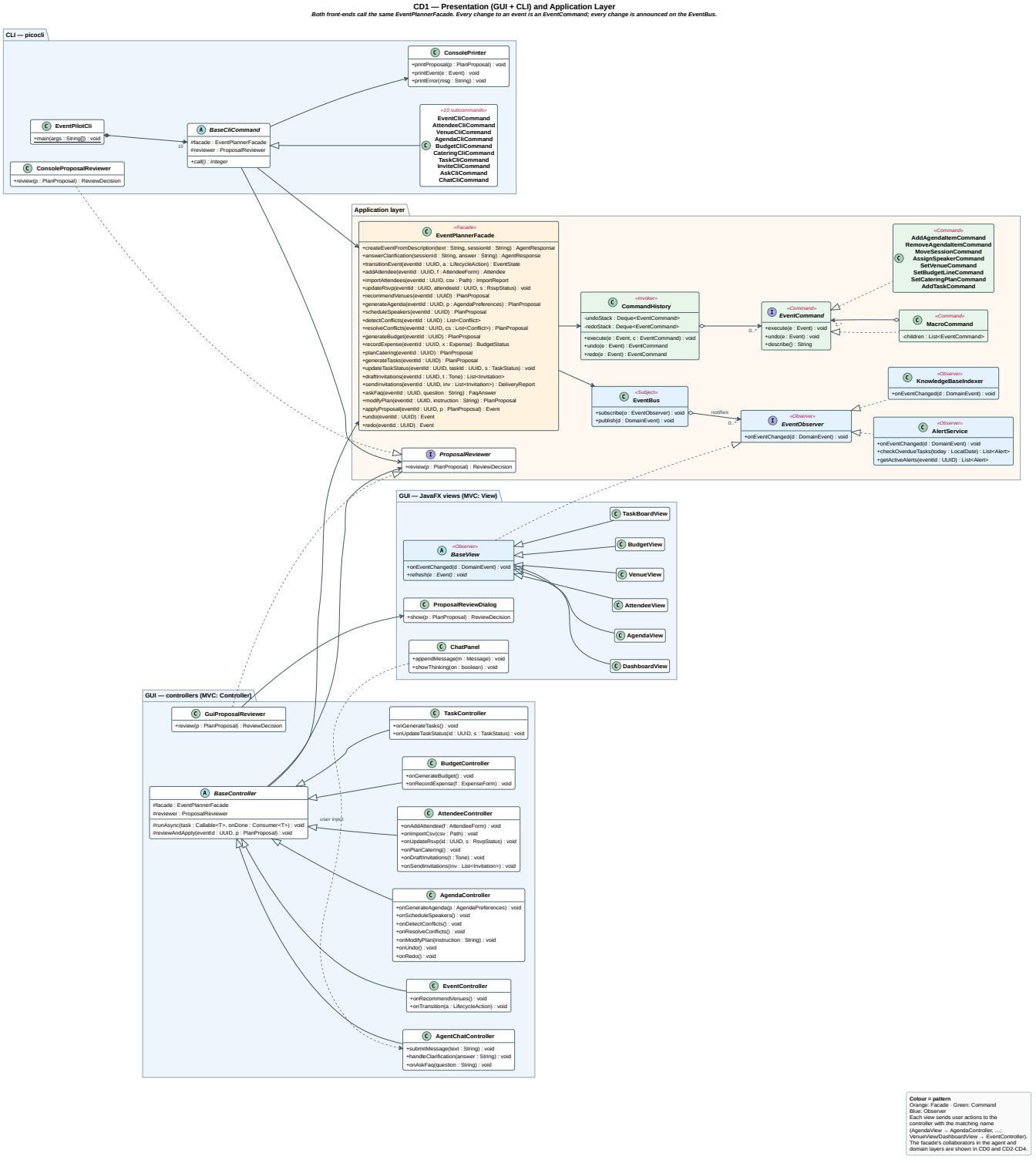



*Figure 3 — GUI views and controllers (MVC), the CLI, the facade, Command (`EventCommand`, `CommandHistory`, `MacroCommand`) and Observer (`EventBus`, `EventObserver`). Both front-ends depend only on `EventPlannerFacade` and the `ProposalReviewer` interface.*

## 3.3 CD2 — Agent Layer

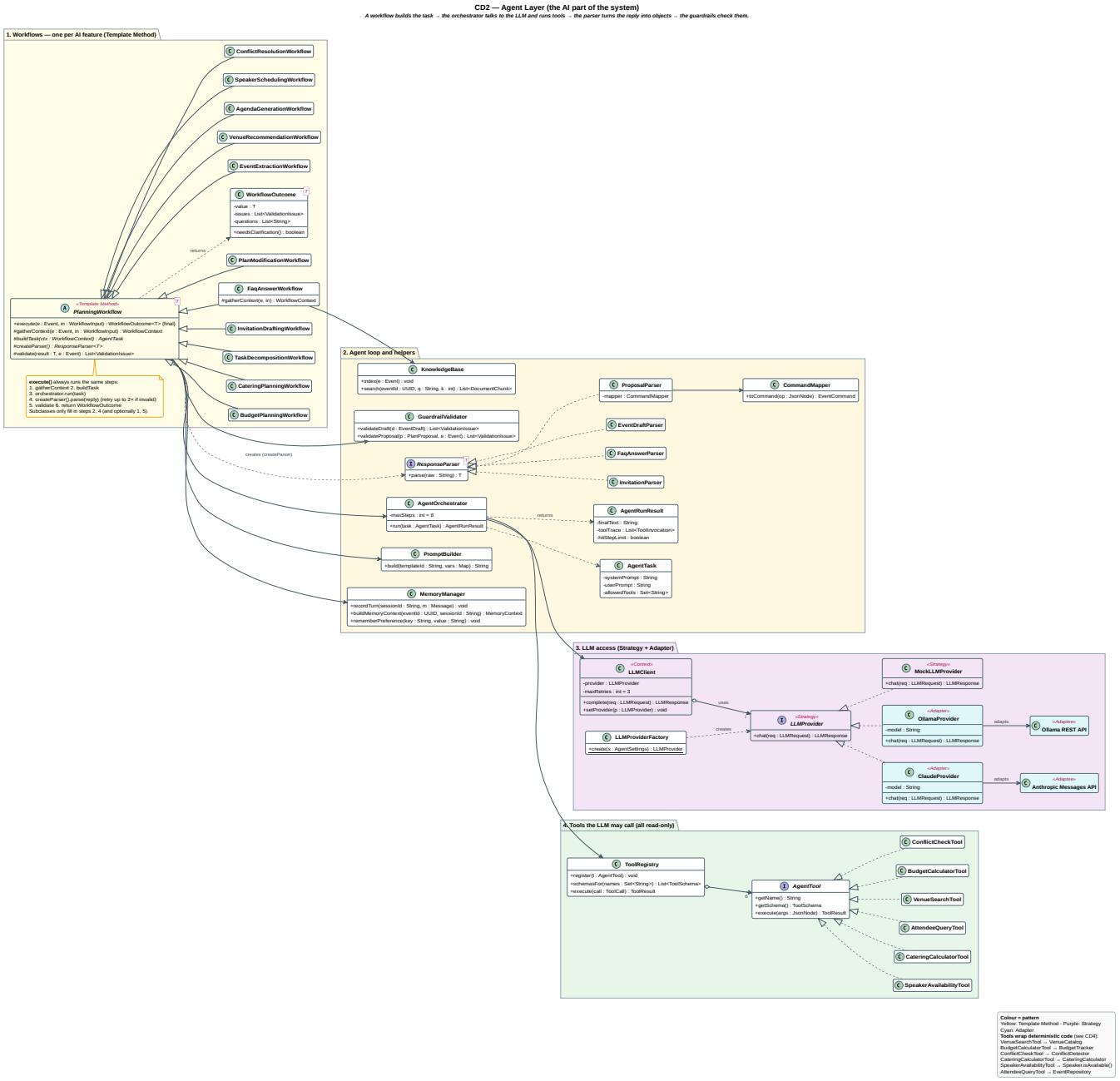



*Figure 4 — The AI components. `PlanningWorkflow<T>` fixes the steps of every AI feature; 11 subclasses fill in the details. `AgentOrchestrator` runs the tool loop. `LLMProvider` is the swappable model, and `ToolRegistry` holds the 6 tools.*

## 3.4 CD3 — Domain Model

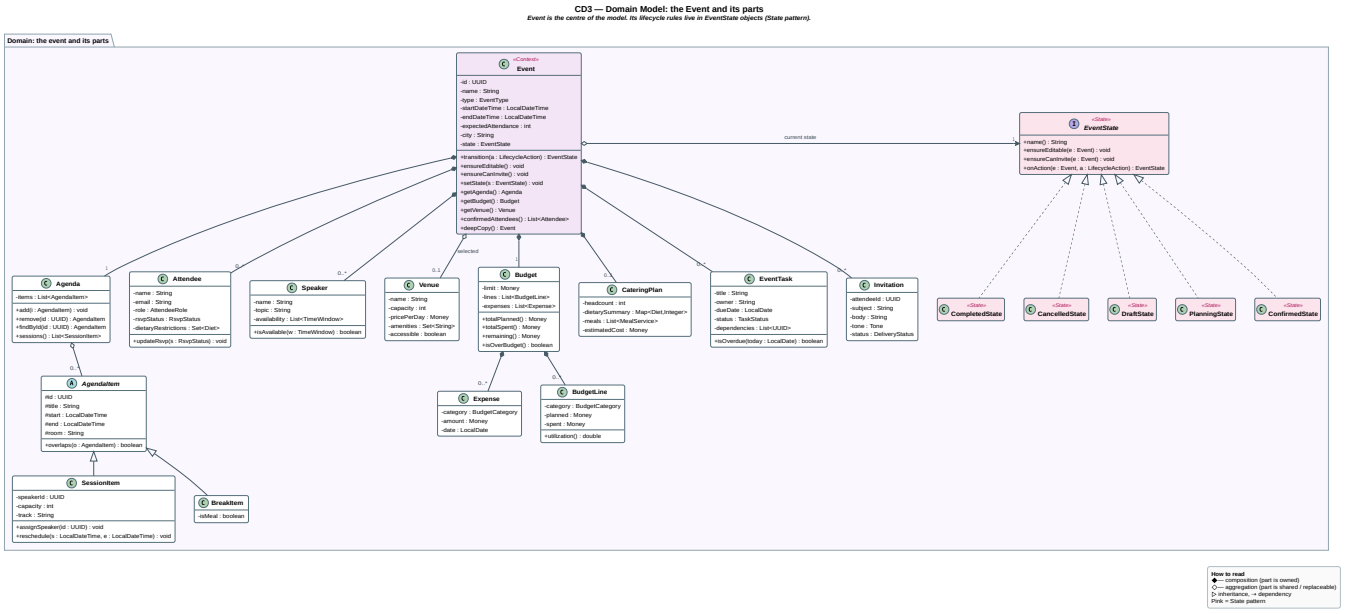



*Figure 5 — `Event` owns its agenda, attendees, speakers, budget, catering plan, tasks and invitations, and shares a `Venue` from the catalog. It delegates lifecycle rules to its current `EventState`.*

## 3.5 CD4 — Domain Services, Agent Results and Infrastructure

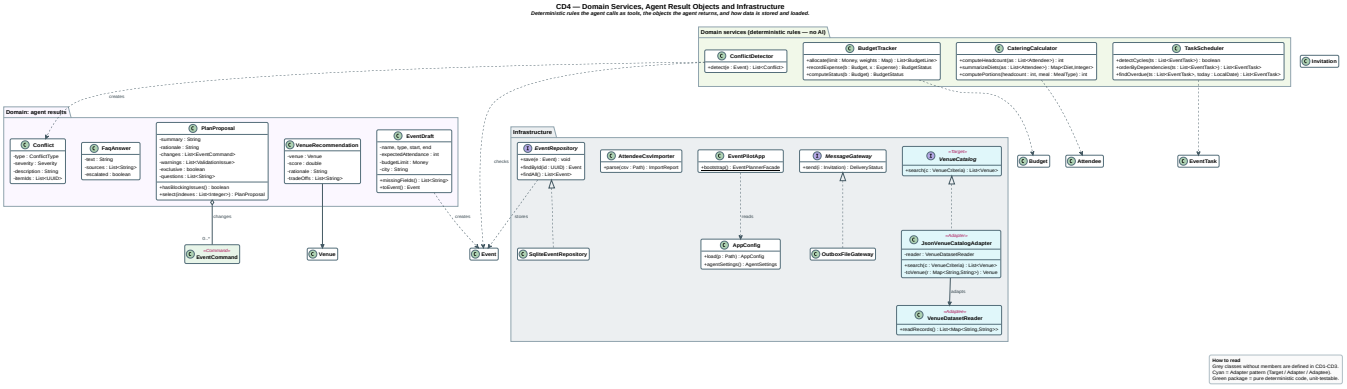



*Figure 6 — Deterministic rule engines (called by the agent's tools), the objects the agent returns, and storage and adapters.*

## 3.6 Key Classes and Responsibilities

| Class | Responsibility |
|---|---|
| `EventPlannerFacade` | The single entry point for every user goal; coordinates repository, workflows, commands and the event bus |
| `BaseController` / `BaseCliCommand` | Turn GUI events or CLI arguments into facade calls; `runAsync` keeps the GUI responsive during AI calls |
| `ProposalReviewer` | Interface for the human approval step (`GuiProposalReviewer` shows a dialog, `ConsoleProposalReviewer` prints a y/N prompt) |
| `EventCommand`, `MacroCommand`, `CommandHistory` | Represent each change as an object; execute, undo and redo |
| `EventBus`, `EventObserver` | Tell views, `AlertService` and `KnowledgeBaseIndexer` that an event changed |
| `PlanningWorkflow<T>` (+11) | Run one AI feature: context → prompt → agent → parse → validate |
| `AgentOrchestrator` | The agent loop: call the LLM, run requested tools, feed results back, stop after 8 steps |
| `LLMClient`, `LLMProvider` | Retries and time-outs; provider-specific API calls (Claude, Ollama, Mock) |
| `ToolRegistry`, `AgentTool` (+6) | Expose tools to the LLM, check arguments against a JSON schema, and run them (read-only) |
| `MemoryManager`, `KnowledgeBase` | Conversation and preference memory; searchable event information for the FAQ |
| `GuardrailValidator` | Deterministic checks on drafts and proposals |

| Class | Responsibility |
|---|---|
| `ResponseParser<T>`, `CommandMapper` | Turn LLM JSON into domain objects or commands |
| `Event`, `EventState` (+5) | The event and its lifecycle rules |
| `ConflictDetector`, `BudgetTracker`, `CateringCalculator`, `TaskScheduler` | Rule engines (pure Java, easy to unit-test) |
| `EventRepository`, `VenueCatalog`, `MessageGateway`, `AttendeeCsvImporter`, `AppConfig` | Storage, venue data, e-mail, CSV import, configuration |

### 3.7 Important Relationships and Multiplicities

| Relationship | Kind | Multiplicity | Why |
|---|---|---|---|
| `Event` ◆ `Agenda`, `Budget` | Composition | 1 – 1 | Have no meaning without their event |
| `Event` ◆ `Attendee`, `Speaker`, `EventTask`, `Invitation` | Composition | 1 – 0..* | Owned by one event |
| `Event` ◇ `Venue` | Aggregation | 0..* – 0..1 | A venue exists in the catalog independently and can be reused |
| `Event` ◇ `EventState` | Aggregation | 1 – 1 | Exactly one current state, replaced on each transition |
| `Agenda` ◇ `AgendaItem` | Aggregation | 1 – 0..* | Items can be moved and removed |
| `MacroCommand` ◇ `EventCommand` | Aggregation | 1 – 1..* | One approved proposal = one undoable unit |
| `EventBus` ◇ `EventObserver` | Aggregation | 1 – 0..* | Observers subscribe at start-up |
| `LLMClient` ◇ `LLMProvider` | Aggregation | 1 – 1 | Provider can be replaced at run time |
| `ToolRegistry` ◇ `AgentTool` | Aggregation | 1 – 6 | Tools registered at start-up |
| `PlanningWorkflow` ⇢ `ResponseParser` | Dependency (creates) | — | Each workflow creates the parser for its output type |

**Design principles applied:** separation of concerns (UI, application, agent, domain, infrastructure); single responsibility (e.g. `ConflictDetector` only detects, `ConflictResolutionWorkflow` only proposes); open/closed (a new AI feature is a new workflow subclass, and a new model is a new provider); dependency inversion (the facade and workflows depend on interfaces such as `EventRepository`, `LLMProvider`, `VenueCatalog`, `MessageGateway` and `ProposalReviewer`, which `EventPilotApp.bootstrap()` wires to real or test implementations).

---

## 4. Design Pattern Explanations

Seven patterns are used. Each solves a specific problem in this system. MVC is used as the overall GUI architecture (views + controllers + model behind the facade) but is not counted among the seven.

| Pattern | Participating classes | Problem addressed | Features |
|---|---|---|---|
| Facade | `EventPlannerFacade` | Two front-ends, many subsystems | All |
| Command | `EventCommand`, concrete commands, `MacroCommand`, `CommandHistory` | Review AI changes before applying; undo/redo | F03–F09, F12 |
| Observer | `EventBus`, `EventObserver`, views, `AlertService`, `KnowledgeBaseIndexer` | Keep many parts in sync after a change | F02, F06, F07, F09, F11 |
| Strategy | `LLMClient`, `LLMProvider`, `ClaudeProvider`, `OllamaProvider`, `MockLLMProvider` | Swap AI models without code changes | All AI features |
| Template Method | `PlanningWorkflow<T>` + 11 subclasses | Same reliable steps for every AI feature | All AI features |
| State | `Event`, `EventState`, 5 state classes | What is allowed depends on the event's stage | F10, UC15, all editing |
| Adapter | `JsonVenueCatalogAdapter`, `ClaudeProvider`, `OllamaProvider` | External data and APIs have the wrong shape | F03, all AI features |

### P1 — Facade

- **Problem.** Each user goal touches many objects: the repository, a state check, a workflow, the guardrails, the command history and the event bus. The same goals must be available from both the GUI and the CLI. Without a single entry point, this coordination code would be copied into every controller and every CLI command.
- **Participants and roles.** `EventPlannerFacade` is the **facade**, with one method per user goal. The repository, workflows, `CommandHistory`, `EventBus`, `GuardrailValidator`, `ConflictDetector`, `BudgetTracker`, `TaskScheduler`, `AttendeeCsvImporter` and `MessageGateway` are the **subsystem** classes. The GUI controllers and CLI commands are the **clients**.
- **Why it fits.** The GUI and CLI get exactly the same small API, so the CLI is guaranteed to offer the same major functions as the GUI. The facade is also the natural place for the "load → change → save → notify" sequence.
- **Without it.** Duplicated logic in about 20 controller methods and 10 CLI commands, with an easy chance that the CLI forgets a step (for example, not re-validating before applying).

### P2 — Command

- **Problem.** AI changes must be **shown to the organizer before they happen**, it must be possible to accept only some of them, and every change must be **undoable**. The LLM's free-form answer must also be limited to a safe, fixed set of operations.
- **Participants and roles.** `EventCommand` is the **command** interface (`execute`, `undo`, `describe`). `AddAgendaItemCommand`, `RemoveAgendaItemCommand`, `MoveSessionCommand`, `AssignSpeakerCommand`, `SetVenueCommand`, `SetBudgetLineCommand`, `SetCateringPlanCommand` and `AddTaskCommand` are **concrete commands**; each stores what it needs to undo itself. `MacroCommand` groups an approved proposal. `CommandHistory` is the **invoker** that keeps the undo and redo stacks. `Event` is the **receiver**. `CommandMapper` and the facade are the **clients** that create commands.
- **Why it fits.** A proposal becomes a list of objects that can be displayed, selected, executed and undone. The AI can only request operations that exist as command classes.
- **Without it.** The agent would change the event directly, which is unsafe and impossible to review. Undo would need full copies of the event or custom reverse code for each feature.

### P3 — Observer

- **Problem.** One change, such as a moved session, must update several unrelated parts: the agenda and dashboard views, the alert service (are there new conflicts?), and the FAQ knowledge base (attendee answers must reflect the new time). The facade should not have to know about all of them.
- **Participants and roles.** `EventBus` is the **subject** (`subscribe`, `publish`). `EventObserver` is the **observer** interface. `BaseView` and its subclasses, `AlertService` and `KnowledgeBaseIndexer` are **concrete observers**. `DomainEvent` carries what changed.
- **Why it fits.** New listeners can be added without touching the facade, the views always show current data, and the FAQ never answers from stale information.
- **Without it.** The facade would call every view and service by hand. Forgetting one would cause stale screens, missed alerts or wrong FAQ answers, and the application layer would depend on JavaFX.

### P4 — Strategy

- **Problem.** The agent must work with different models: Claude for quality, Ollama for free offline use, and a scripted mock for tests. The model should be chosen by configuration.
- **Participants and roles.** `LLMClient` is the **context**; it holds a provider and adds retries and time-outs. `LLMProvider` is the **strategy** interface (`chat`). `ClaudeProvider`, `OllamaProvider` and `MockLLMProvider` are **concrete strategies**. `LLMProviderFactory` picks one from `AppConfig`.
- **Why it fits.** Workflows never know which model they use. Stage 3 tests can use the mock for repeatable results and a real model for behavior tests.
- **Without it.** `if (provider == "claude") … else …` in every LLM call; tests would always need the internet and would not be repeatable.

### P5 — Template Method

- **Problem.** Eleven AI features need the **same reliable procedure**: gather context → build prompt → run the agent → parse JSON → fix invalid JSON → validate → return. Only the prompt, the parser and some checks differ. Copying this procedure 11 times would lead to inconsistent error handling.
- **Participants and roles.** `PlanningWorkflow<T>` is the **abstract class**. Its `execute()` is `final` and defines the steps. `buildTask()` and `createParser()` are abstract; `gatherContext()` and `validate()` are hooks with default behavior. The 11 **concrete classes** (`EventExtractionWorkflow`, `VenueRecommendationWorkflow`, `AgendaGenerationWorkflow`, `SpeakerSchedulingWorkflow`, `ConflictResolutionWorkflow`, `BudgetPlanningWorkflow`, `CateringPlanningWorkflow`, `TaskDecompositionWorkflow`, `InvitationDraftingWorkflow`, `FaqAnswerWorkflow`, `PlanModificationWorkflow`) fill in the steps. For example, `FaqAnswerWorkflow` overrides `gatherContext()` to do retrieval, and `ConflictResolutionWorkflow` overrides `validate()` to test the fix on a copy.
- **Why it fits.** Safety features (repair retries, guardrails, tool trace) are written once and every AI feature gets them. A new AI feature only needs its prompt and parser.
- **Without it.** Eleven near-identical methods, where a bug fixed in one would stay broken in the others.

### P6 — State

- **Problem.** What the organizer may do depends on the event's stage. A *Draft* cannot send invitations; a *Completed* or *Cancelled* event is read-only; *Confirm* requires a venue and an agenda. Writing `if (status == …)` checks in every facade method would scatter these rules everywhere.
- **Participants and roles.** `Event` is the **context**; it holds the current state and forwards `ensureEditable()`, `ensureCanInvite()` and `transition()` to it. `EventState` is the **state** interface. `DraftState`, `PlanningState`, `ConfirmedState`, `CompletedState` and `CancelledState` are the **concrete states**; each knows its permissions and its allowed next states.
- **Why it fits.** Each stage's rules are in one class, transitions are explicit and easy to test, and a new stage (e.g. *Postponed*) is a new class.
- **Without it.** Large condition blocks repeated in many methods, with a real risk of, for example, sending invitations for a cancelled event.

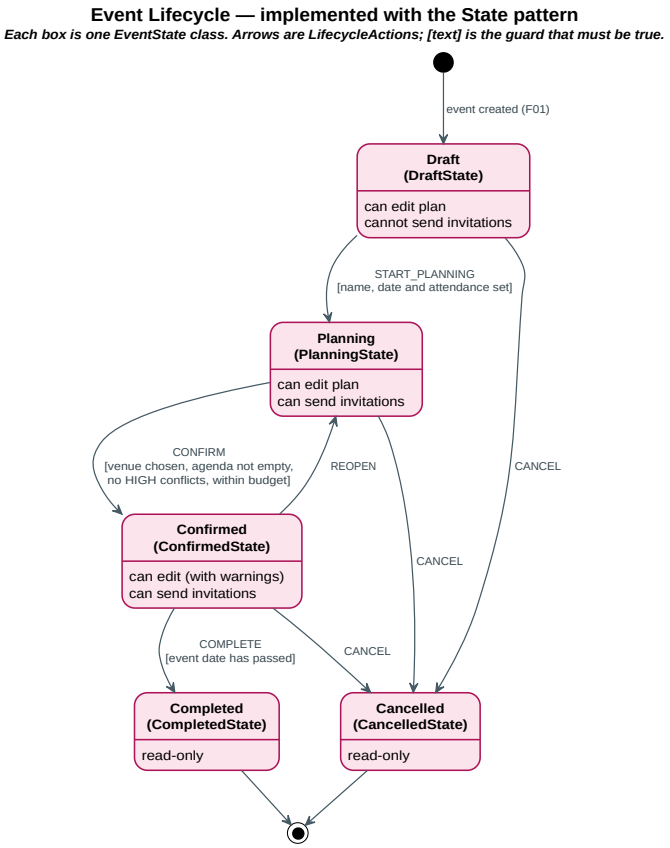



*Figure 7 — The event lifecycle implemented by the state classes.*

## P7 — Adapter

- **Problem.** External sources have their own formats. The venue dataset uses different field names than our `Venue` class, and the Anthropic and Ollama APIs each use their own request and response JSON. The rest of the system should see one stable interface.

**Participants and roles:**

| Target (what our code expects) | Adapter | Adaptee (what exists) |
|---|---|---|
| `VenueCatalog.search(criteria)` | `JsonVenueCatalogAdapter` | `VenueDatasetReader.readRecords()` |
| `LLMProvider.chat(request)` | `ClaudeProvider` | Anthropic Messages API |
| `LLMProvider.chat(request)` | `OllamaProvider` | Ollama REST API |

Clients: `VenueSearchTool`, `LLMClient`.

- **Why it fits.** Vendor formats are isolated in one class each. Replacing the venue dataset with a real venue API later means writing one new adapter.
- **Without it.** Vendor JSON handling would spread through tools and workflows, and an API change would break many classes.

---

## 5. Use-Case Diagram

The diagram is shown in two views to keep it readable. View 1 shows the people who use the system and every use case. View 2 shows the external systems the use cases depend on.

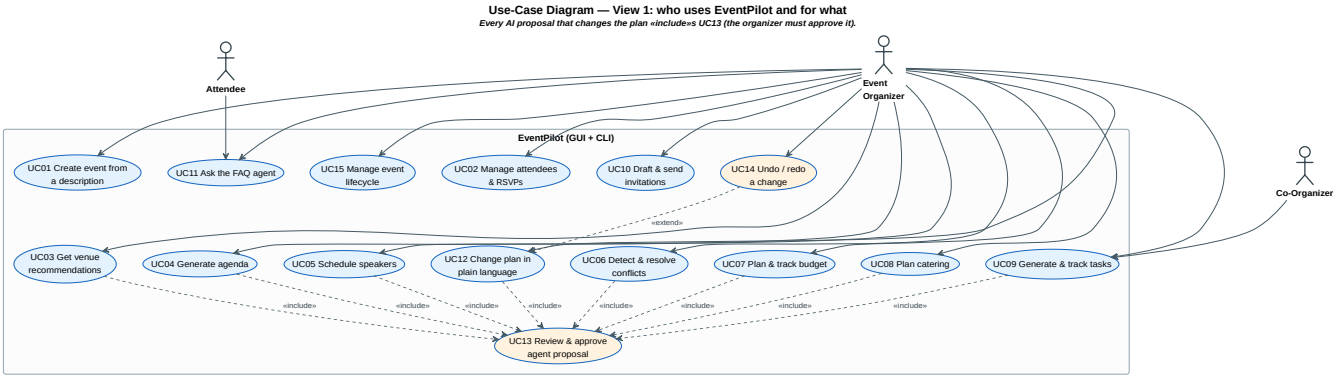



*Figure 8 — View 1: human actors, all 15 use cases, «include» and «extend».*

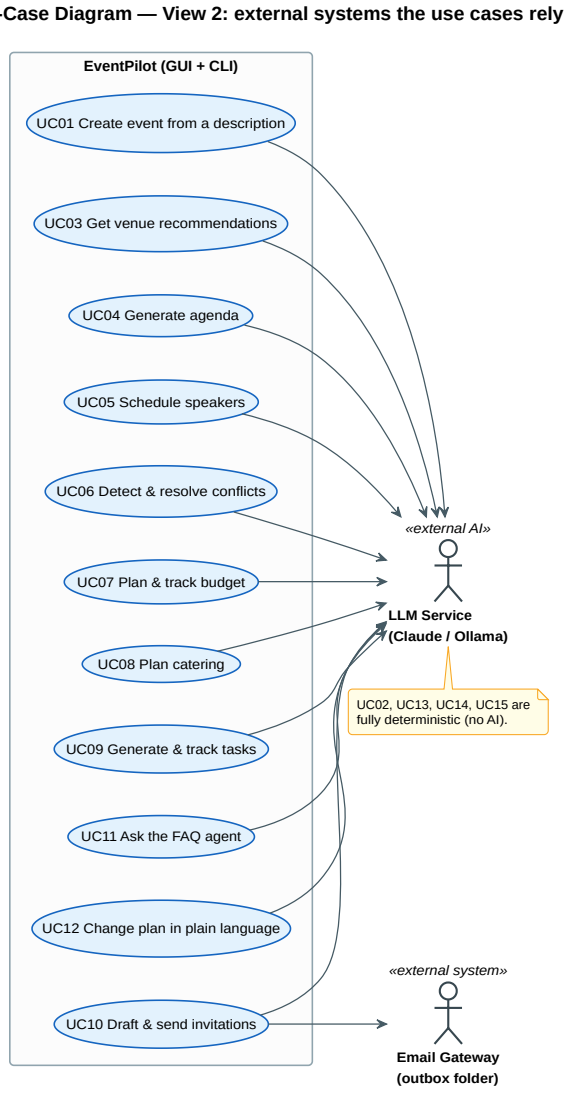



*Figure 9 — View 2: external actors (LLM service, e-mail gateway).*

### Actors

| Actor | Type | Role |
|---|---|---|
| Event Organizer | Primary | Plans the event, reviews and approves agent proposals |

| Actor | Type | Role |
|---|---|---|
| Attendee | Secondary | Asks questions through the FAQ agent |
| Co-Organizer | Secondary | Updates tasks assigned to them |
| LLM Service (Claude / Ollama) | External AI | Reasoning, planning, tool selection, text generation |
| Email Gateway (outbox folder) | External system | Delivers invitations |

**Relationships:** UC03–UC09 and UC12 «include» **UC13 Review & approve agent proposal**, because no AI change is applied without approval. **UC14 Undo/redo** «extend»s UC12 (and can be used after any applied change).

**Feature coverage:** UC01–UC12 correspond one-to-one to F01–F12. UC13, UC14 and UC15 are supporting use cases (approval, undo, lifecycle).

---

## 6. Detailed Use-Case Descriptions

**Shared precondition (P0):** EventPilot is running (GUI or CLI) and the database is available. **Shared exception (E-AI):** for AI use cases, if the LLM cannot be reached after 3 retries, the user is told that the AI is unavailable, nothing changes, and a manual alternative is offered where one exists.

### UC01 — Create Event from a Description

| Field | Description |
|---|---|
| **ID / Name** | UC01 — Create Event from a Description |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | Create a correctly specified event without filling a long form |
| **Preconditions** | P0 |
| **Trigger** | Organizer types a description in the chat (or `event create "<text>"`) |
| **Main Success Scenario** | 1. Organizer enters the description. 2. System asks the agent to extract the event details. 3. System validates the result. 4. All required details are present. 5. System creates the event in *Draft*, saves it and shows it on the Dashboard. 6. System confirms with a summary. |
| **Alternative / Exception Flows** | 4a. Details missing or contradictory → system asks follow-up questions; organizer answers; continue at step 2. 3a. Agent output invalid → system asks the agent to correct it (max 2). 1a. Not an event request → system explains what it can do. E-AI → manual *New Event* form. |
| **Postconditions** | A new event exists in Draft state (or nothing is created). |
| **Related Feature** | F01 |

### UC02 — Manage Attendees & RSVPs

| Field | Description |
|---|---|
| **ID / Name** | UC02 — Manage Attendees & RSVPs |
| **Actor(s)** | Event Organizer |
| **Goal** | Keep an accurate attendee list with roles, diets and RSVP status |
| **Preconditions** | P0; event exists and is not Completed/Cancelled |
| **Trigger** | Organizer adds an attendee, imports a CSV, or changes an RSVP |
| **Main Success Scenario** | 1. Organizer chooses *Import CSV* and a file. 2. System checks every row. 3. System adds the valid, non-duplicate attendees. 4. System saves and refreshes the views. 5. System shows the import report. 6. Organizer changes an RSVP; system saves and updates the counts. |
| **Alternative / Exception Flows** | 1a. Manual add through the form. 2a. File missing or wrong header → error, nothing imported. 2b. Some rows invalid → those rows are listed; the rest import. 3a. Duplicate e-mail → skipped and reported. |
| **Postconditions** | Attendee list and RSVP counts are up to date; FAQ data refreshed. |
| **Related Feature** | F02 |

### UC03 — Get Venue Recommendations

| Field | Description |
|---|---|
| **ID / Name** | UC03 — Get Venue Recommendations |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | Choose a venue that fits capacity, budget, location and accessibility |
| **Preconditions** | P0; event is editable and has an attendance estimate |
| **Trigger** | Organizer clicks *Recommend venues* (or `venue recommend`) |
| **Main Success Scenario** | 1. Organizer asks for recommendations. 2. Agent searches the venue catalog with the venue tool. 3. Agent checks the venue budget with the budget tool. 4. Agent ranks up to three venues with reasons and trade-offs. 5. System validates them. 6. «include» UC13: organizer picks one. 7. System sets the venue and saves. |
| **Alternative / Exception Flows** | 2a. Invalid tool arguments → returned to the agent to correct. 2b. No match → agent relaxes only price or district and says so; otherwise reports "no suitable venue". 5a. Venue not from the catalog or too small → blocked. 6a. Organizer rejects → no change. E-AI. |
| **Postconditions** | Venue selected (undoable), or unchanged. |

| Field | Description |
|---|---|
| **Related Feature** | F03 |

## UC04 — Generate Agenda

| Field | Description |
|---|---|
| **ID / Name** | UC04 — Generate Agenda |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | Get a complete agenda without conflicts that matches the organizer's preferences |
| **Preconditions** | P0; event editable; event hours set |
| **Trigger** | Organizer clicks *Generate agenda* and confirms preferences |
| **Main Success Scenario** | 1. Organizer sets preferences. 2. Agent drafts the agenda, checking it with the conflict tool. 3. System converts the draft to commands and validates them. 4. «include» UC13. 5. System applies the changes as one undoable step, saves and notifies. |
| **Alternative / Exception Flows** | 1a. Agenda already exists → choose replace or extend. 3a. Items outside event hours or overlapping → warnings; serious ones block approval. 4a. Rejected → no change. E-AI. |
| **Postconditions** | Agenda saved; change can be undone. |
| **Related Feature** | F04 |

## UC05 — Schedule Speakers

| Field | Description |
|---|---|
| **ID / Name** | UC05 — Schedule Speakers |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | Place every speaker in a session they are available for, with no double-booking |
| **Preconditions** | P0; event editable; speakers and sessions exist |
| **Trigger** | Organizer clicks *Auto-schedule speakers* |
| **Main Success Scenario** | 1. Organizer requests scheduling. 2. Agent reads speaker availability with the availability tool. 3. Agent proposes assignments and checks them with the conflict tool until no serious conflict remains. 4. System validates. 5. «include» UC13. 6. System applies and saves. |
| **Alternative / Exception Flows** | 1a. No speakers or sessions → error before any AI call. 3a. Impossible constraints → proposal for the speakers that fit, plus an explanation. E-AI. |
| **Postconditions** | Speakers assigned (undoable). |
| **Related Feature** | F05 |

## UC06 — Detect & Resolve Conflicts

| Field | Description |
|---|---|
| **ID / Name** | UC06 — Detect & Resolve Conflicts |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | Find all scheduling problems and fix them with as few changes as possible |
| **Preconditions** | P0; event exists |
| **Trigger** | Organizer clicks *Check conflicts* (also runs automatically after every change) |
| **Main Success Scenario** | 1. System runs the conflict detector. 2. System lists and highlights conflicts. 3. Organizer clicks *Ask agent to fix*. 4. Agent proposes changes. 5. System tests the fix on a copy of the event. 6. «include» UC13. 7. System applies the fix. |
| **Alternative / Exception Flows** | 2a. No conflicts → "No conflicts found". 5a. Fix creates new conflicts → blocked and retried once; then the remaining conflicts are listed for manual fixing. E-AI → detection still works (no AI needed). |
| **Postconditions** | Conflicts shown; if approved, resolved. |
| **Related Feature** | F06 |

## UC07 — Plan & Track Budget

| Field | Description |
|---|---|
| **ID / Name** | UC07 — Plan & Track Budget |
| **Actor(s)** | Event Organizer; LLM Service |

| Field | Description |
|---|---|
| **Goal** | Split the budget sensibly and stay within it |
| **Preconditions** | P0; event has a budget limit |
| **Trigger** | Organizer clicks *Generate budget plan* or records an expense |
| **Main Success Scenario** | 1. Organizer asks for a plan. 2. Agent chooses the split; the budget tool calculates the amounts. 3. System checks the total does not exceed the limit. 4. «include» UC13. 5. Organizer records an expense. 6. System validates it, updates totals and saves. 7. Alert service checks spending. |
| **Alternative / Exception Flows** | 1a. No limit → organizer asked to set one. 6a. Invalid amount or category → form error. 7a. Category over plan or ≥ 90 % spent → alert on the Dashboard. E-AI (planning only; tracking still works). |
| **Postconditions** | Budget lines, expenses and alerts up to date. |
| **Related Feature** | F07 |

## UC08 — Plan Catering

| Field | Description |
|---|---|
| **ID / Name** | UC08 — Plan Catering |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | A catering plan that fits the headcount, covers every diet and stays within budget |
| **Preconditions** | P0; event editable; attendees exist |
| **Trigger** | Organizer clicks *Plan catering* |
| **Main Success Scenario** | 1. System computes headcount and dietary counts. 2. Agent designs meals and menus; the catering tool computes portions. 3. System checks diets are covered and cost fits. 4. «include» UC13. 5. System saves the plan. |
| **Alternative / Exception Flows** | 1a. No RSVPs yet → expected attendance used, with a warning. 3a. A diet not covered → blocked, retried once. 3b. Over budget → warning. E-AI. |
| **Postconditions** | Catering plan saved; FAQ can answer food questions. |
| **Related Feature** | F08 |

## UC09 — Generate & Track Tasks

| Field | Description |
|---|---|
| **ID / Name** | UC09 — Generate & Track Tasks |
| **Actor(s)** | Event Organizer; Co-Organizer; LLM Service |
| **Goal** | A complete, correctly ordered task list that stays up to date |
| **Preconditions** | P0; event has a date |
| **Trigger** | Organizer clicks *Generate task plan*; co-organizer moves a task card |
| **Main Success Scenario** | 1. Organizer requests a task plan. 2. Agent lists tasks with owners, deadlines and dependencies. 3. System checks for circular dependencies and orders the tasks. 4. «include» UC13. 5. Co-organizer marks a task done. 6. System checks its dependencies are done and saves. |
| **Alternative / Exception Flows** | 3a. Cycle or deadline after the event → blocked, retried once. 6a. Dependency not done → refused ("blocked by ..."). Overdue tasks → alert at start-up and every morning. |
| **Postconditions** | Tasks and statuses saved; overdue alerts current. |
| **Related Feature** | F09 |

## UC10 — Draft & Send Invitations

| Field | Description |
|---|---|
| **ID / Name** | UC10 — Draft & Send Invitations |
| **Actor(s)** | Event Organizer; LLM Service; Email Gateway |
| **Goal** | Send correct, personalised invitations |
| **Preconditions** | P0; event in *Planning* or *Confirmed*; attendees exist |
| **Trigger** | Organizer chooses a tone and clicks *Draft invitations* |
| **Main Success Scenario** | 1. System checks the event state allows invitations. 2. Agent drafts one template per attendee group. 3. System checks each contains the real date, time and venue. 4. Organizer edits and clicks *Send*. 5. System sends each invitation through the gateway. 6. System shows the delivery report. |

| Field | Description |
|---|---|
| **Alternative / Exception Flows** | 1a. Wrong state → refused with guidance. 3a. Wrong or missing facts → regenerated. 5a. One delivery fails → recorded; the rest continue. E-AI → organizer can write the text manually. |
| **Postconditions** | Invitations recorded with delivery status. |
| **Related Feature** | F10 |

## UC11 — Ask the FAQ Agent

| Field | Description |
|---|---|
| **ID / Name** | UC11 — Ask the FAQ Agent |
| **Actor(s)** | Attendee (or Organizer); LLM Service |
| **Goal** | Get a correct, sourced answer about the event |
| **Preconditions** | P0; event information is indexed |
| **Trigger** | User asks a question in FAQ mode (or `ask "<question>"`) |
| **Main Success Scenario** | 1. User asks a question. 2. System finds the 5 most relevant pieces of event information. 3. Agent answers only from them and cites them. 4. System checks the cited sources were retrieved. 5. System shows the answer with sources. |
| **Alternative / Exception Flows** | 2a. Nothing relevant → "forwarded to the organizer" plus an organizer alert. 4a. Unknown source cited → safe reply, escalated. 1a. Off-topic or manipulation attempt → declined. |
| **Postconditions** | Answer shown, or question escalated. |
| **Related Feature** | F11 |

## UC12 — Change Plan in Plain Language

| Field | Description |
|---|---|
| **ID / Name** | UC12 — Change Plan in Plain Language |
| **Actor(s)** | Event Organizer; LLM Service |
| **Goal** | Change the plan by describing the change |
| **Preconditions** | P0; event editable |
| **Trigger** | Organizer types an instruction (or `agenda modify "<text>"`) |
| **Main Success Scenario** | 1. Organizer enters the instruction. 2. Agent maps it to operations on real items. 3. System turns the operations into commands and validates them. 4. «include» UC13. 5. System applies them as one undoable step. |
| **Alternative / Exception Flows** | 2a. Ambiguous → agent asks which item. 3a. Unknown item or operation → agent asked to fix it. 5a. Organizer reverts → «extend» UC14. E-AI. |
| **Postconditions** | Plan changed (undoable), or unchanged. |
| **Related Feature** | F12 |

## UC13 — Review & Approve Agent Proposal (included)

| Field | Description |
|---|---|
| **ID / Name** | UC13 — Review & Approve Agent Proposal |
| **Actor(s)** | Event Organizer |
| **Goal** | Keep the organizer in control of every AI change |
| **Preconditions** | A workflow has produced a `PlanProposal` |
| **Trigger** | Included by UC03–UC09 and UC12 |
| **Main Success Scenario** | 1. System shows the changes, reasons and warnings. 2. Organizer approves all. 3. System re-checks the proposal against the current event. 4. System applies all changes as one undoable step, saves and notifies. |
| **Alternative / Exception Flows** | 2a. Approve selected (or pick one of several alternatives). 2b. Reject → nothing changes. 3a. A blocking problem appears (e.g. the event changed meanwhile) → refused with reasons. CLI: printed list and a `[y/N/select]` prompt. |
| **Postconditions** | Changes applied, or event unchanged. |
| **Related Feature** | F03–F09, F12 |

## UC14 — Undo / Redo a Change

| Field | Description |
|---|---|
| **ID / Name** | UC14 — Undo / Redo a Change |
| **Actor(s)** | Event Organizer |
| **Goal** | Reverse (or re-apply) the latest change |
| **Preconditions** | P0; event editable; at least one change in the history |
| **Trigger** | *Undo* / *Redo* button, Ctrl+Z / Ctrl+Y, or `agenda undo` / `agenda redo` |
| **Main Success Scenario** | 1. Organizer clicks *Undo*. 2. System undoes the last change (all its steps, in reverse order). 3. System saves and refreshes, showing "Undid: ...". |
| **Alternative / Exception Flows** | 2a. Nothing to undo → message. Redo works the same way in the other direction. A new change after undo clears the redo history. |
| **Postconditions** | Event restored to its previous state. |
| **Related Feature** | F12 (and any applied proposal) |

## UC15 — Manage Event Lifecycle

| Field | Description |
|---|---|
| **ID / Name** | UC15 — Manage Event Lifecycle |
| **Actor(s)** | Event Organizer |
| **Goal** | Move the event through Draft → Planning → Confirmed → Completed (or Cancelled) |
| **Preconditions** | P0; event exists |
| **Trigger** | Organizer clicks *Start planning*, *Confirm*, *Reopen*, *Complete* or *Cancel* |
| **Main Success Scenario** | 1. Organizer clicks *Confirm*. 2. System checks there are no serious conflicts and the event is within budget. 3. The current state checks its own rule (venue chosen, agenda not empty). 4. The event moves to *Confirmed*; system saves and updates the badge. |
| **Alternative / Exception Flows** | 2a/3a. Check fails, or the move is not allowed from this state → refused with reasons. Cancel → asks for confirmation first. |
| **Postconditions** | Event in its new state; allowed actions change (e.g. invitations become possible). |
| **Related Feature** | Enables F10 and controls editing in all features |

---

## 7. Sequence Diagrams

Ten sequence diagrams cover all important behaviors. Each one uses the classes and methods of the class diagram. Participants are grouped by layer (GUI/CLI, Application, Agent, Domain, Infrastructure), steps are numbered, and notes explain the key steps in plain words. The tool loop (SD02) and the review-and-apply sequence (SD03, Part 2) are shown in full once; the other diagrams refer to them.

| SD | Shows | Features | Use cases |
|---|---|---|---|
| SD01 | Create event from a description | F01 | UC01 |
| SD02 | Venue recommendation — **the agent tool loop** | F03 | UC03 |
| SD03 | Agenda generation + **review & apply** | F04 | UC04, UC13 |
| SD04 | Speaker scheduling, conflict detection & resolution | F05, F06 | UC05, UC06 |
| SD05 | Budget plan, expenses and alerts | F07 | UC07 |
| SD06 | Attendee import, RSVP, catering plan | F02, F08 | UC02, UC08 |
| SD07 | Task plan, updates, overdue alerts | F09 | UC09 |
| SD08 | Lifecycle change + invitations (**State pattern**) | F10 | UC15, UC10 |
| SD09 | FAQ agent with retrieval | F11 | UC11 |
| SD10 | Plain-language change + undo, **from the CLI** | F12 | UC12, UC14 |

### SD01 — Create Event from a Description

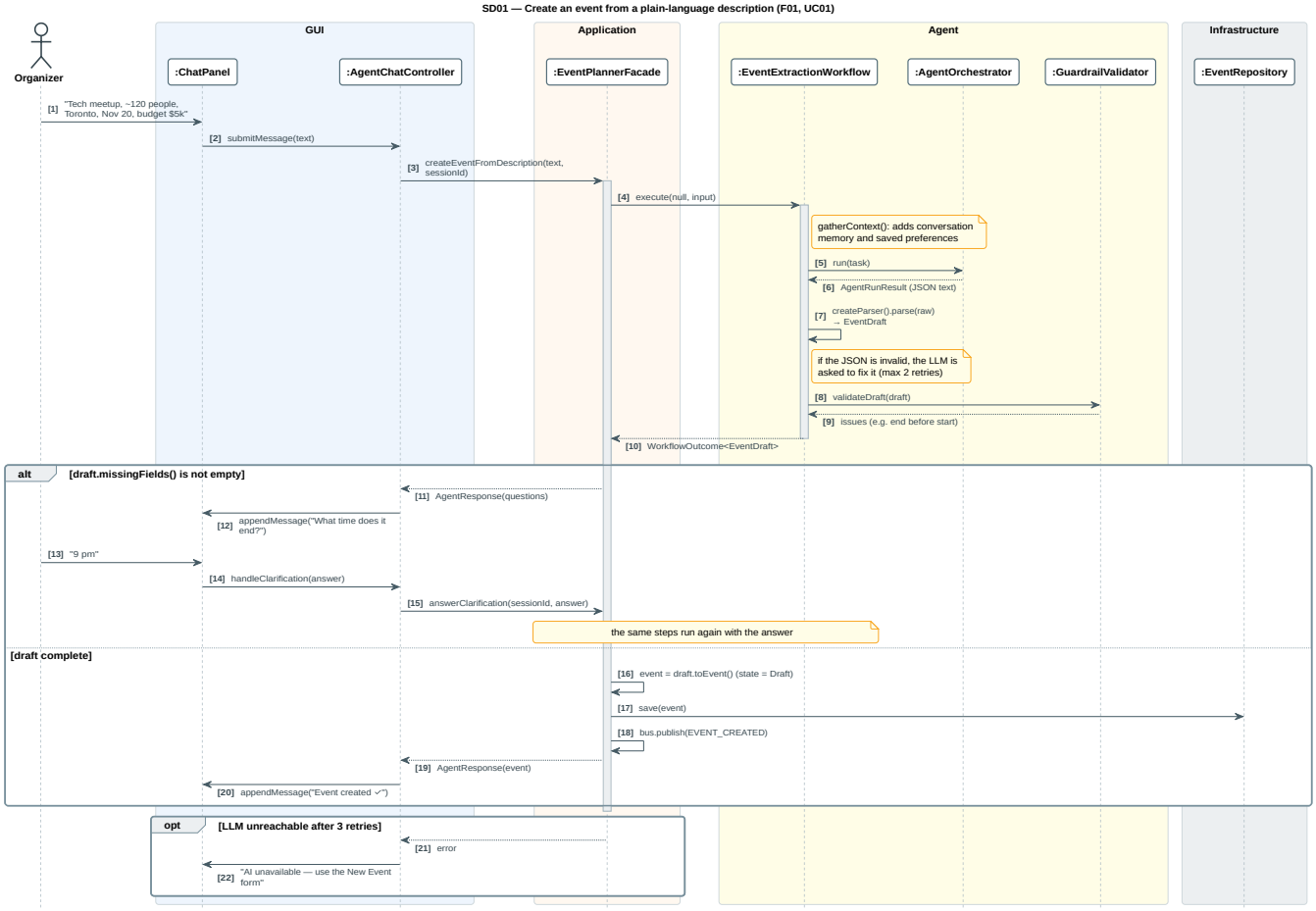



*Figure 10.* The facade runs `EventExtractionWorkflow`, which uses the agent, parses the reply into an `EventDraft` and validates it. If information is missing, the organizer is asked and the steps repeat. Otherwise the event is created in Draft state and saved. The `opt` fragment shows what happens when the AI is unavailable.

## SD02 — Venue Recommendation: the Agent Tool Loop

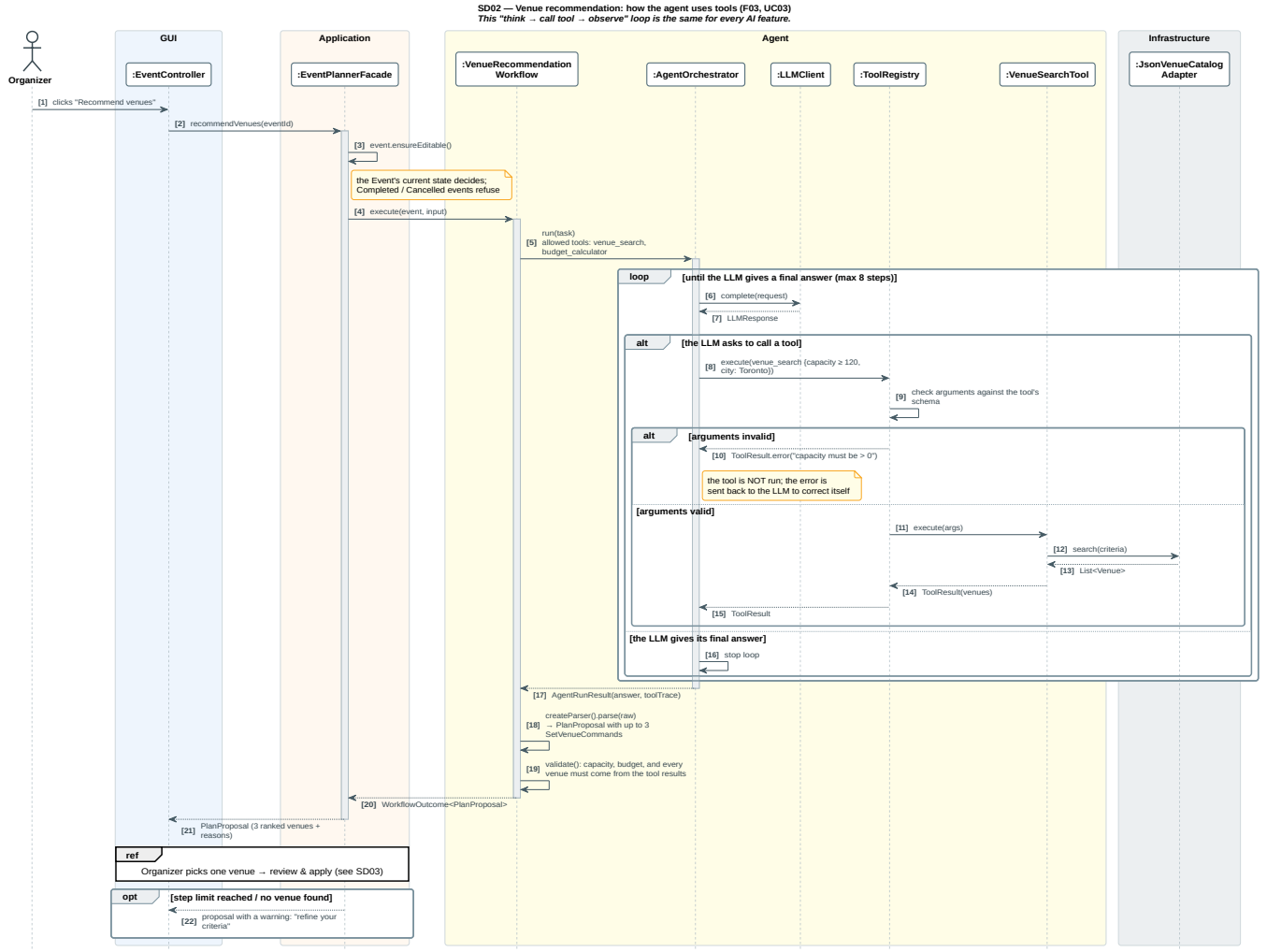



*Figure 11.* This diagram shows how the agent works. Inside the loop, the LLM either requests a tool or gives its final answer. `ToolRegistry` checks the arguments **before** running a tool; invalid arguments are sent back to the LLM without running anything. `VenueSearchTool` uses the venue adapter. The final answer is parsed into a proposal and validated: a venue the tool never returned is rejected.

## SD03 — Agenda Generation, then Review & Apply

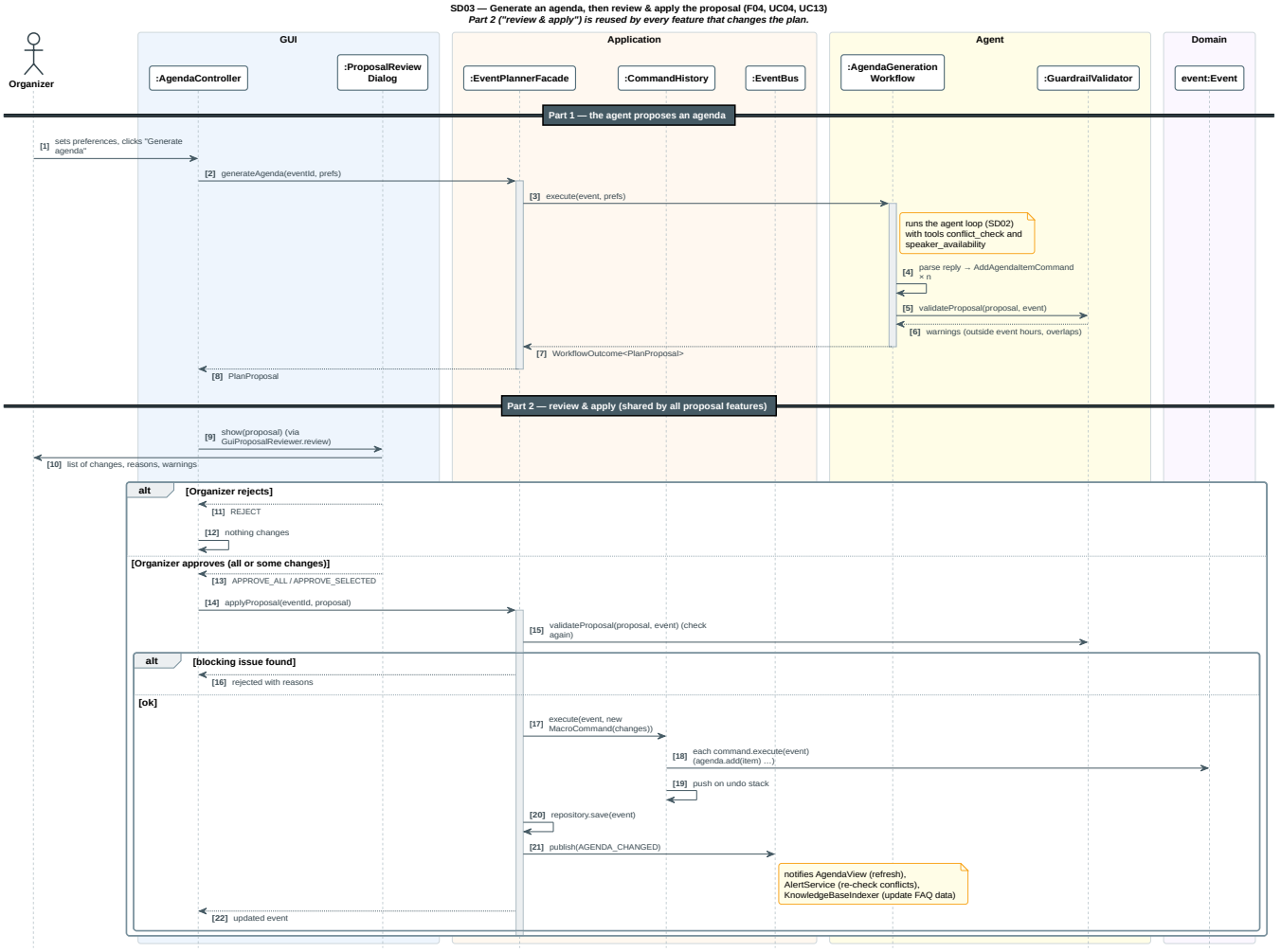



*Figure 12.* Part 1: the agent proposes `AddAgendaItemCommand`s. Part 2 (used by every proposal feature): the dialog shows the proposal; on approval the facade re-checks it, `CommandHistory` executes one `MacroCommand`, the event is saved, and `EventBus` notifies the views, `AlertService` and `KnowledgeBaseIndexer`.

## SD04 — Speakers and Conflicts

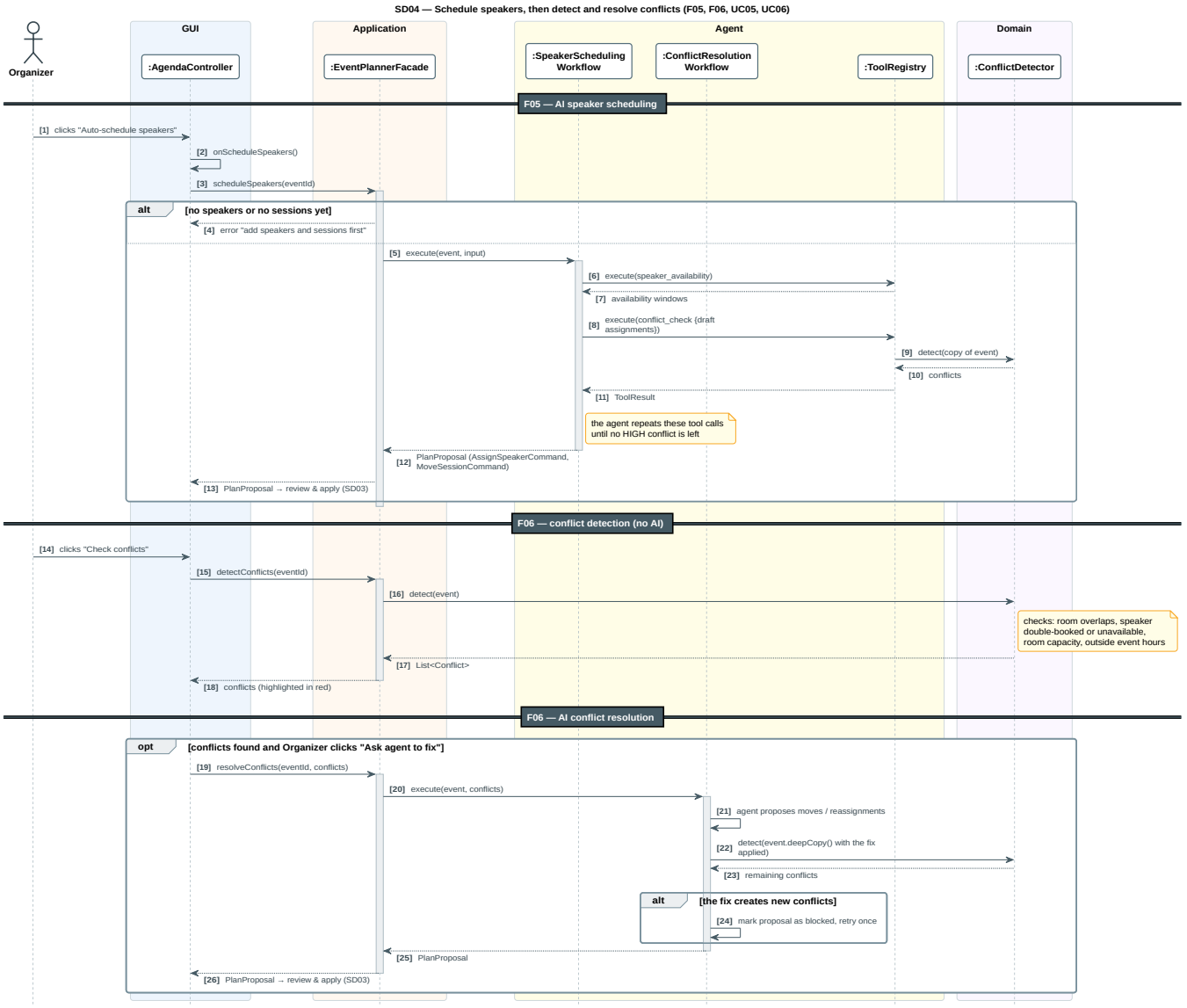



*Figure 13.* Scheduling uses the availability and conflict tools in a loop. Detection is plain code (`ConflictDetector.detect`). Resolution is proposed by the agent and tested on `event.deepCopy()`, so a fix that creates new conflicts is blocked.

## SD05 — Budget and Alerts

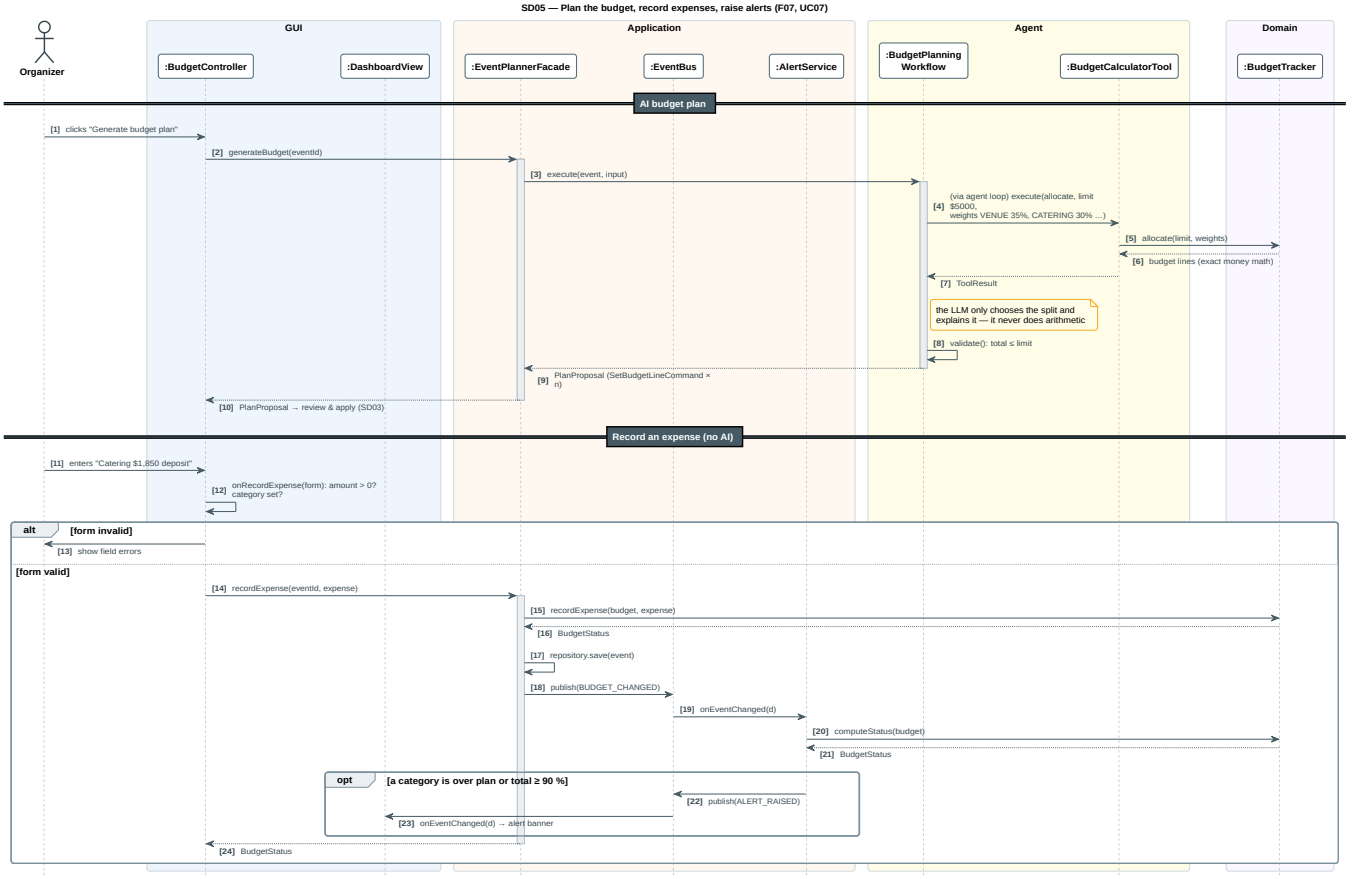



*Figure 14.* The LLM chooses the split, but `BudgetTracker.allocate()` does the math. Recording an expense is deterministic. `AlertService`, an observer, raises an alert that reaches the Dashboard through the bus.

## SD06 — Attendees and Catering

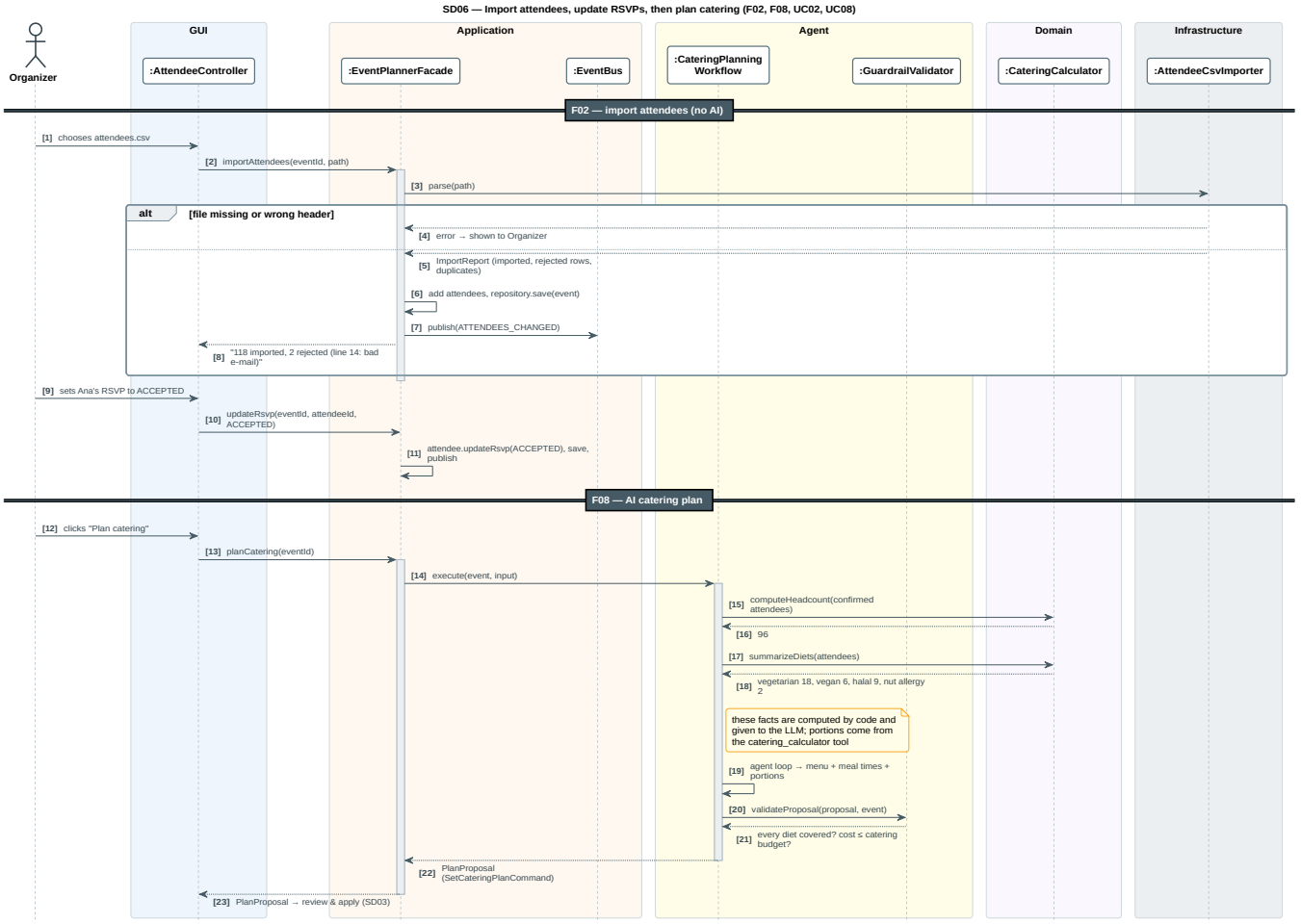



*Figure 15.* The CSV import validates each row and reports problems. For catering, the headcount and dietary counts are computed by `CateringCalculator` and given to the agent as facts. The guardrails check that every diet is covered and the cost fits.

## SD07 — Tasks

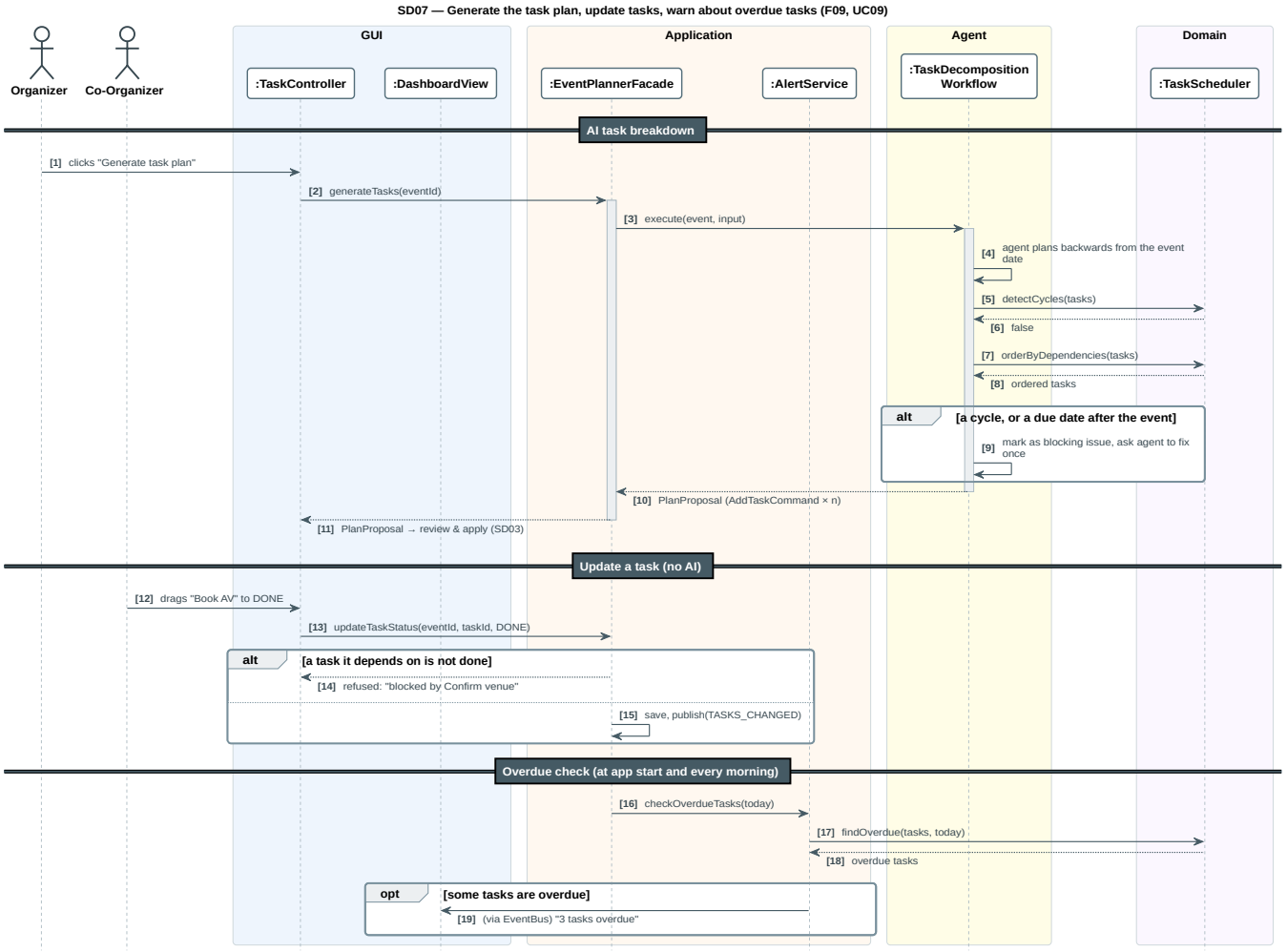



*Figure 16.* The agent's task list is checked by `TaskScheduler` (cycles, ordering). A status change is refused if a dependency is not finished. The overdue check runs at start-up and every morning.

## SD08 — Lifecycle Change and Invitations (State Pattern)

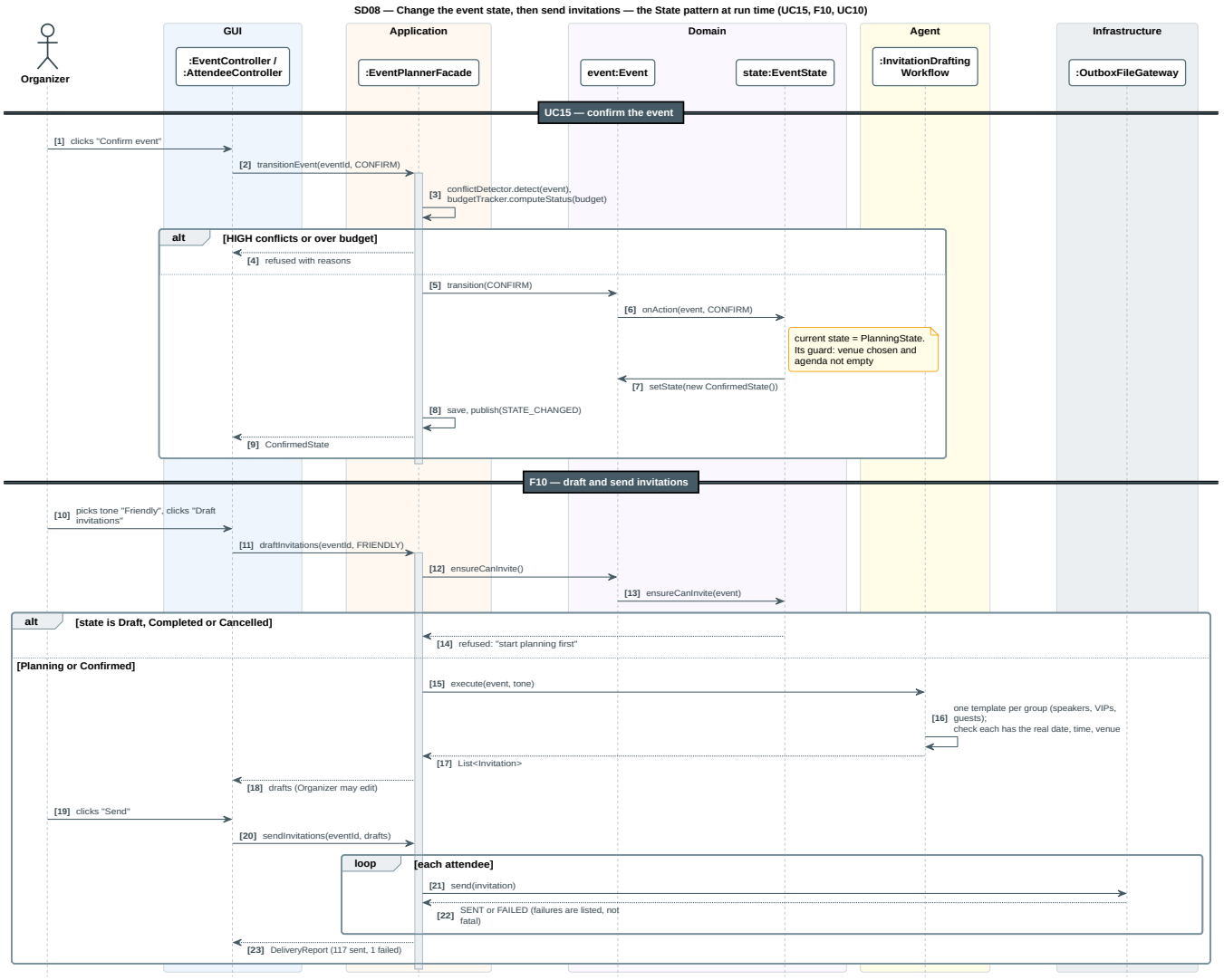



*Figure 17.* `Event.transition(CONFIRM)` is handled by the current state object (`PlanningState`), which checks its rule and installs a new `ConfirmedState`. Later, `ensureCanInvite()` is again answered by the state object. Sending continues even if one delivery fails.

## SD09 — FAQ Agent with Retrieval

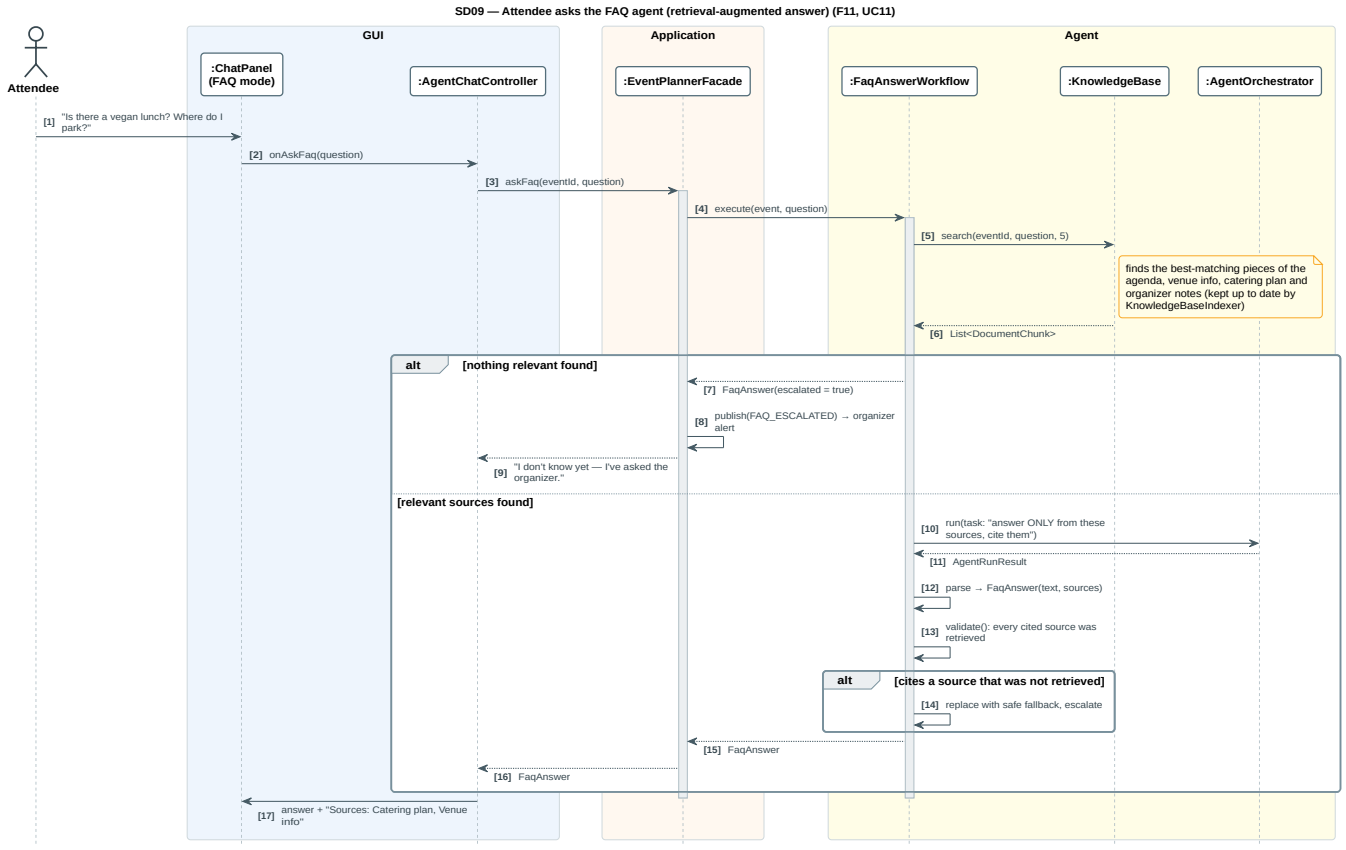



*Figure 18.* `KnowledgeBase.search()` finds relevant event information. If nothing is relevant, the question is escalated **without** asking the LLM. Otherwise the LLM may answer only from the sources, and the cited sources are checked.

## SD10 — Plain-Language Change and Undo from the CLI

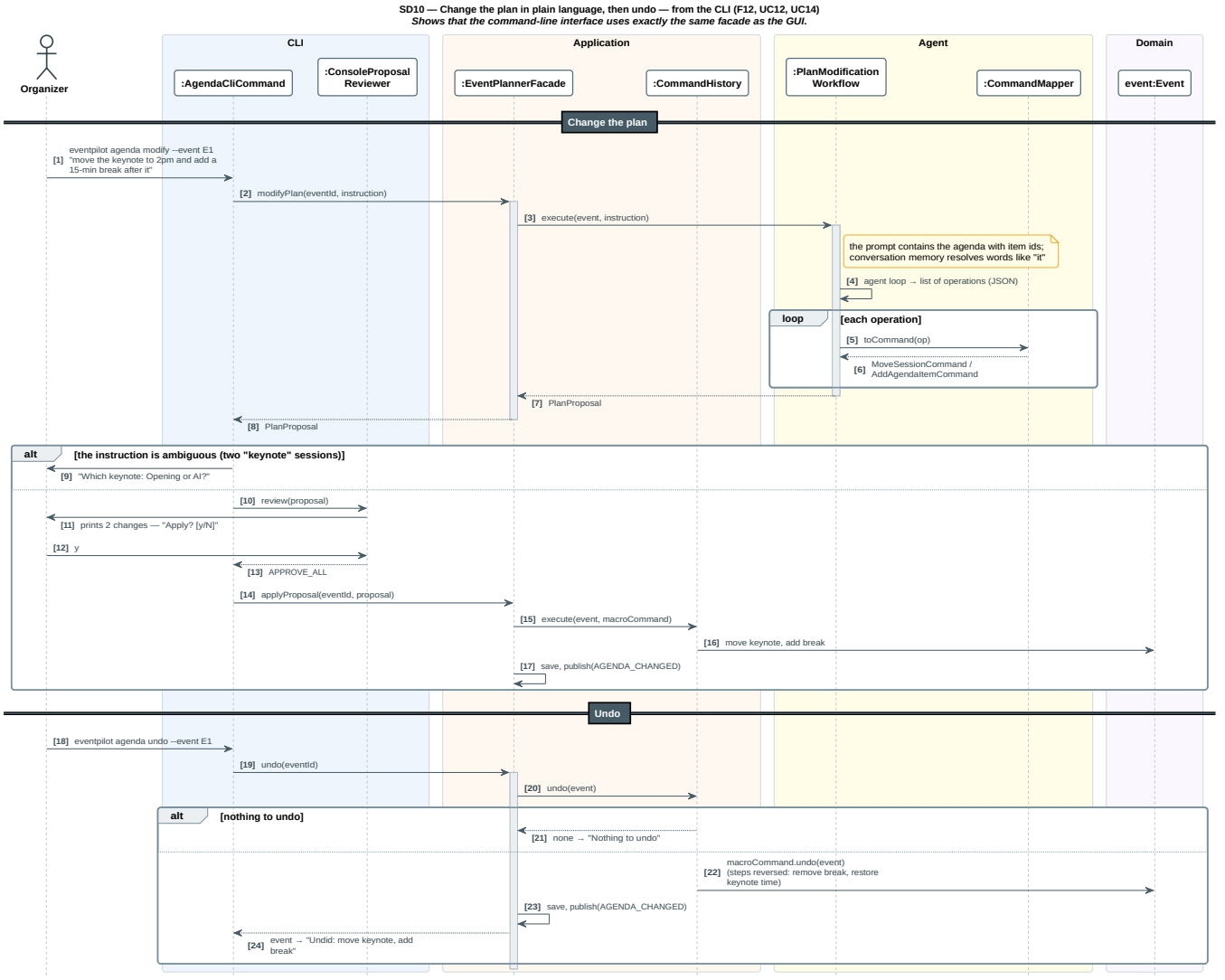



*Figure 19.* The CLI calls the same facade methods as the GUI. `CommandMapper` turns each operation into a typed command; `ConsoleProposalReviewer` asks `y/N`. Undo calls `MacroCommand.undo()`, which reverses the steps in the opposite order.

## 8. Feature-to-Design Traceability Table

| Feature | Description | Type | Use Case | Classes | Key Methods | Sequence Diagram | Design Pattern(s) |
|---|---|---|---|---|---|---|---|
| **F01** | Create event from a description | AI + validation | UC01 | `ChatPanel`, `AgentChatController`, `EventPlannerFacade`, `EventExtractionWorkflow`, `AgentOrchestrator`, `EventDraftParser`, `GuardrailValidator`, `EventDraft`, `Event`, `EventRepository` | `submitMessage()`, `createEventFromDescription()`, `answerClarification()`, `execute()`, `run()`, `parse()`, `validateDraft()`, `missingFields()`, `toEvent()`, `save()` | SD01 | Facade, Template Method, Strategy |
| **F02** | Attendee & RSVP management | Deterministic | UC02 | `AttendeeView`, `AttendeeController`, `EventPlannerFacade`, `AttendeeCsvImporter`, `Attendee`, `EventBus`, `KnowledgeBaseIndexer` | `onImportCsv()`, `onUpdateRsvp()`, `importAttendees()`, `updateRsvp()`, `parse()`, `Attendee.updateRsvp()`, `publish()` | SD06 | Facade, Observer, State (edit check) |
| **F03** | Venue recommendation | Hybrid | UC03, UC13 | `VenueView`, `EventController`, `EventPlannerFacade`, `VenueRecommendationWorkflow`, `AgentOrchestrator`, `LLMClient`, `ToolRegistry`, `VenueSearchTool`, `JsonVenueCatalogAdapter`, `GuardrailValidator`, `SetVenueCommand` | `onRecommendVenues()`, `recommendVenues()`, `ensureEditable()`, `run()`, `complete()`, `ToolRegistry.execute()`, `search()`, `validateProposal()`, `applyProposal()` | SD02 | Template Method, Strategy, Adapter, Command, State |
| **F04** | Agenda generation | Hybrid | UC04, UC13 | `AgendaView`, `AgendaController`, `ProposalReviewDialog`, `GuiProposalReviewer`, `EventPlannerFacade`, `AgendaGenerationWorkflow`, `ProposalParser`, `CommandMapper`, `GuardrailValidator`, `CommandHistory`, `MacroCommand`, `AddAgendaItemCommand`, `Agenda`, `EventBus` | `onGenerateAgenda()`, `generateAgenda()`, `toCommand()`, `validateProposal()`, `review()`, `show()`, `applyProposal()`, `CommandHistory.execute()`, `Agenda.add()`, `publish()` | SD03 | Template Method, Command, Observer, Facade |
| **F05** | Speaker scheduling | Hybrid | UC05, UC13 | `AgendaController`, `EventPlannerFacade`, `SpeakerSchedulingWorkflow`, `ToolRegistry`, `SpeakerAvailabilityTool`, `ConflictCheckTool`, `ConflictDetector`, `AssignSpeakerCommand`, `MoveSessionCommand`, `Speaker` | `onScheduleSpeakers()`, `scheduleSpeakers()`, `ToolRegistry.execute()`, `isAvailable()`, `detect()`, `assignSpeaker()`, `reschedule()` | SD04 | Template Method, Command |
| **F06** | Conflict detection & resolution | Hybrid | UC06, UC13 | `AgendaView`, `AgendaController`, `EventPlannerFacade`, `ConflictDetector`, `Conflict`, `ConflictResolutionWorkflow`, `Event`, `AlertService` | `onDetectConflicts()`, `detectConflicts()`, `detect()`, `onResolveConflicts()`, `resolveConflicts()`, `validate()`, `deepCopy()` | SD04 | Template Method, Command, Observer |
| **F07** | Budget planning & tracking | Hybrid | UC07, UC13 | `BudgetView`, `BudgetController`, `EventPlannerFacade`, `BudgetPlanningWorkflow`, `BudgetCalculatorTool`, `BudgetTracker`, `SetBudgetLineCommand`, `Budget`, `EventBus`, `AlertService`, `DashboardView` | `onGenerateBudget()`, `generateBudget()`, `allocate()`, `onRecordExpense()`, `recordExpense()`, `computeStatus()`, `onEventChanged()` | SD05 | Template Method, Command, Observer |

| Feature | Description | Type | Use Case | Classes | Key Methods | Sequence Diagram | Design Pattern(s) |
|---|---|---|---|---|---|---|---|
| **F08** | Catering plan | Hybrid | UC08, UC13 | `AttendeeController`, `EventPlannerFacade`, `CateringPlanningWorkflow`, `CateringCalculator`, `CateringCalculatorTool`, `GuardrailValidator`, `SetCateringPlanCommand`, `CateringPlan` | `onPlanCatering()`, `planCatering()`, `computeHeadcount()`, `summarizeDiets()`, `computePortions()`, `validateProposal()` | SD06 | Template Method, Command |
| **F09** | Task plan & tracking | Hybrid | UC09, UC13 | `TaskBoardView`, `TaskController`, `EventPlannerFacade`, `TaskDecompositionWorkflow`, `TaskScheduler`, `AddTaskCommand`, `EventTask`, `AlertService` | `onGenerateTasks()`, `generateTasks()`, `detectCycles()`, `orderByDependencies()`, `onUpdateTaskStatus()`, `updateTaskStatus()`, `checkOverdueTasks()`, `findOverdue()` | SD07 | Template Method, Command, Observer |
| **F10** | Invitations | AI + delivery | UC10, UC15 | `AttendeeController`, `EventController`, `EventPlannerFacade`, `Event`, `EventState`, `PlanningState`, `ConfirmedState`, `InvitationDraftingWorkflow`, `InvitationParser`, `Invitation`, `MessageGateway`, `OutboxFileGateway` | `onDraftInvitations()`, `draftInvitations()`, `ensureCanInvite()`, `onSendInvitations()`, `sendInvitations()`, `send()`, `transitionEvent()`, `transition()`, `onAction()`, `setState()` | SD08 | **State**, Template Method, Facade |
| **F11** | FAQ agent (RAG) | AI | UC11 | `ChatPanel`, `AgentChatController`, `EventPlannerFacade`, `FaqAnswerWorkflow`, `KnowledgeBase`, `AgentOrchestrator`, `FaqAnswerParser`, `FaqAnswer`, `KnowledgeBaseIndexer` | `onAskFaq()`, `askFaq()`, `gatherContext()`, `search()`, `index()`, `run()`, `parse()`, `validate()` | SD09 | Template Method, Observer, Strategy |
| **F12** | Plain-language changes + undo/redo | Hybrid | UC12, UC13, UC14 | `AgendaCliCommand`, `ConsoleProposalReviewer`, `AgendaController`, `EventPlannerFacade`, `PlanModificationWorkflow`, `CommandMapper`, `CommandHistory`, `MacroCommand`, `MoveSessionCommand`, `AddAgendaItemCommand` | `call()`, `onModifyPlan()`, `modifyPlan()`, `toCommand()`, `review()`, `applyProposal()`, `undo()`, `redo()`, `MacroCommand.undo()` | SD10 | **Command**, Template Method, Facade |

**Supporting use cases:** UC13 Review & approve → `ProposalReviewer`, `GuiProposalReviewer`, `ConsoleProposalReviewer`, `ProposalReviewDialog`, `CommandHistory` (SD03 Part 2). UC14 Undo/redo → `CommandHistory.undo()/redo()` (SD10). UC15 Lifecycle → `Event.transition()`, `EventState.onAction()` (SD08).

---

## 9. Feature Implementation Explanations

All features that change the plan end with the same **review & apply** step (SD03, Part 2): `BaseController.reviewAndApply()` → `ProposalReviewer.review()` → `EventPlannerFacade.applyProposal()` → `GuardrailValidator.validateProposal()` → `CommandHistory.execute(MacroCommand)` → `EventRepository.save()` → `EventBus.publish()`.

### F01 — Create Event from a Description

**Use case:** UC01 · **Sequence diagram:** SD01

**Classes:** `ChatPanel` (input/output) · `AgentChatController` (forwards messages, runs the call in the background) · `EventPlannerFacade` (coordinates) · `EventExtractionWorkflow` (extraction steps) · `AgentOrchestrator` / `LLMClient` (run the model) · `EventDraftParser` (JSON → `EventDraft`) · `GuardrailValidator` (consistency) · `EventRepository` (save).

**Methods:** `submitMessage()`, `createEventFromDescription()`, `answerClarification()`, `PlanningWorkflow.execute()`, `AgentOrchestrator.run()`, `EventDraftParser.parse()`, `validateDraft()`, `EventDraft.missingFields()`, `EventDraft.toEvent()`, `EventRepository.save()`.

**How it works:** `submitMessage()` passes the text to the facade, which runs `EventExtractionWorkflow.execute()`. The workflow adds conversation memory to the prompt, runs the agent, parses the reply into an `EventDraft`, and validates it. If `missingFields()` is not empty, the questions go back to the chat and the answer returns through `answerClarification()`. When the draft is complete, `toEvent()` creates the event in Draft state, the repository saves it, and the Dashboard refreshes through the event bus.

### F02 — Attendee & RSVP Management

**Use case:** UC02 · **Sequence diagram:** SD06

**Classes:** `AttendeeView`, `AttendeeController`, `EventPlannerFacade`, `AttendeeCsvImporter`, `Attendee`, `EventBus`, `KnowledgeBaseIndexer`.

**Methods:** `onImportCsv()`, `importAttendees()`, `AttendeeCsvImporter.parse()`, `onUpdateRsvp()`, `updateRsvp()`, `Attendee.updateRsvp()`, `EventBus.publish()`.

**How it works:** `importAttendees()` checks that the event is editable, asks `AttendeeCsvImporter.parse()` for an `ImportReport`, adds the valid attendees, saves, and publishes `ATTENDEES_CHANGED`. `AttendeeView` refreshes and `KnowledgeBaseIndexer` re-indexes the FAQ data. RSVP changes take the same path. No AI is involved.

### F03 — Venue Recommendation

**Use case:** UC03 · **Sequence diagram:** SD02

**Classes:** `EventController`, `EventPlannerFacade`, `VenueRecommendationWorkflow`, `AgentOrchestrator`, `LLMClient`, `ToolRegistry`, `VenueSearchTool`, `BudgetCalculatorTool`, `JsonVenueCatalogAdapter`, `ProposalParser`, `GuardrailValidator`, `SetVenueCommand`.

**Methods:** `onRecommendVenues()`, `recommendVenues()`, `Event.ensureEditable()`, `AgentOrchestrator.run()`, `LLMClient.complete()`, `ToolRegistry.execute()`, `VenueCatalog.search()`, `validateProposal()`, `PlanProposal.select()`, `applyProposal()`.

**How it works:** After the state check, the workflow lets the agent use only `venue_search` and `budget_calculator`. In the loop, the LLM requests a search; `ToolRegistry` checks the arguments and runs `VenueSearchTool`, which calls `JsonVenueCatalogAdapter.search()`; the results go back to the LLM. The final ranking is parsed into a proposal of up to three `SetVenueCommand`s, and the guardrails reject venues that are too small, too expensive, or not from the catalog. The organizer picks one, and the chosen command is applied (and can be undone).

### F04 — Agenda Generation

**Use case:** UC04 · **Sequence diagram:** SD03

**Classes:** `AgendaController`, `ProposalReviewDialog`, `GuiProposalReviewer`, `EventPlannerFacade`, `AgendaGenerationWorkflow`, `ProposalParser`, `CommandMapper`, `GuardrailValidator`, `CommandHistory`, `MacroCommand`, `AddAgendaItemCommand`, `Agenda`, `EventBus`.

**Methods:** `onGenerateAgenda()`, `generateAgenda()`, `CommandMapper.toCommand()`, `validateProposal()`, `GuiProposalReviewer.review()`, `ProposalReviewDialog.show()`, `applyProposal()`, `CommandHistory.execute()`, `Agenda.add()`, `EventBus.publish()`.

**How it works:** The workflow gives the agent the event hours, rooms, speakers and preferences, and allows the `conflict_check` tool. The reply is turned into `AddAgendaItemCommand`s by `CommandMapper`, then checked for time-window and overlap problems. The

organizer reviews; on approval the facade re-checks and runs one `MacroCommand`, which adds the items to the `Agenda`. Saving and publishing refreshes the timeline, re-checks conflicts and updates the FAQ data.

### F05 — Speaker Scheduling

**Use case:** UC05 · **Sequence diagram:** SD04

**Classes:** `AgendaController`, `EventPlannerFacade`, `SpeakerSchedulingWorkflow`, `ToolRegistry`, `SpeakerAvailabilityTool`, `ConflictCheckTool`, `ConflictDetector`, `AssignSpeakerCommand`, `MoveSessionCommand`, `Speaker`.

**Methods:** `onScheduleSpeakers()`, `scheduleSpeakers()`, `ToolRegistry.execute()`, `Speaker.isAvailable()`, `ConflictDetector.detect()`, `SessionItem.assignSpeaker()`, `SessionItem.reschedule()`.

**How it works:** The facade first checks that speakers and sessions exist. The agent then reads availability (`speaker_availability` uses `Speaker.isAvailable()`) and tests its draft with `conflict_check`, which runs `ConflictDetector.detect()` on a copy of the event. It repeats until no serious conflict remains. The resulting assign and move commands go through review & apply; each stores the old value so it can be undone.

### F06 — Conflict Detection & Resolution

**Use case:** UC06 · **Sequence diagram:** SD04

**Classes:** `AgendaController`, `EventPlannerFacade`, `ConflictDetector`, `Conflict`, `ConflictResolutionWorkflow`, `Event`, `AlertService`.

**Methods:** `onDetectConflicts()`, `detectConflicts()`, `ConflictDetector.detect()`, `onResolveConflicts()`, `resolveConflicts()`, `ConflictResolutionWorkflow.validate()`, `Event.deepCopy()`.

**How it works:** Detection is pure code: `detect()` checks room overlaps, speaker double-booking and availability, room capacity, and event hours, and the conflicts are highlighted. `AlertService` also runs it after every change. For a fix, `ConflictResolutionWorkflow` asks the agent for changes, and its `validate()` applies them to `event.deepCopy()` and runs `detect()` again. A fix that adds conflicts is blocked and retried once.

### F07 — Budget Planning & Tracking

**Use case:** UC07 · **Sequence diagram:** SD05

**Classes:** `BudgetController`, `EventPlannerFacade`, `BudgetPlanningWorkflow`, `BudgetCalculatorTool`, `BudgetTracker`, `SetBudgetLineCommand`, `Budget`, `EventBus`, `AlertService`, `DashboardView`.

**Methods:** `onGenerateBudget()`, `generateBudget()`, `BudgetTracker.allocate()`, `onRecordExpense()`, `recordExpense()`, `BudgetTracker.recordExpense()`, `computeStatus()`, `AlertService.onEventChanged()`.

**How it works:** The agent chooses the split and calls `budget_calculator`, which uses `BudgetTracker.allocate()` for exact money arithmetic. The `SetBudgetLineCommand`s are checked (total ≤ limit) and applied after review. An expense is validated in the controller, recorded by `BudgetTracker.recordExpense()`, saved and published. `AlertService` then computes the status and raises an alert if a category is over plan or 90 % of the budget is used.

### F08 — Catering Plan

**Use case:** UC08 · **Sequence diagram:** SD06

**Classes:** `AttendeeController`, `EventPlannerFacade`, `CateringPlanningWorkflow`, `CateringCalculator`, `CateringCalculatorTool`, `GuardrailValidator`, `SetCateringPlanCommand`, `CateringPlan`.

**Methods:** `onPlanCatering()`, `planCatering()`, `CateringCalculator.computeHeadcount()`, `summarizeDiets()`, `computePortions()`, `validateProposal()`.

**How it works:** The workflow computes the confirmed headcount and dietary counts in code and gives them to the agent as facts. The agent designs meals aligned with the agenda breaks and gets portions from `catering_calculator`. The proposed `SetCateringPlanCommand` is checked so that every diet is covered and the cost fits the catering budget, then reviewed and applied.

### F09 — Task Plan & Tracking

**Use case:** UC09 · **Sequence diagram:** SD07

**Classes:** `TaskController`, `EventPlannerFacade`, `TaskDecompositionWorkflow`, `TaskScheduler`, `AddTaskCommand`, `EventTask`, `AlertService`, `DashboardView`.

**Methods:** `onGenerateTasks()`, `generateTasks()`, `TaskScheduler.detectCycles()`, `orderByDependencies()`, `onUpdateTaskStatus()`, `updateTaskStatus()`, `AlertService.checkOverdueTasks()`, `findOverdue()`.

**How it works:** The agent plans tasks backwards from the event date. `TaskScheduler` rejects circular dependencies and orders the tasks, and deadlines after the event are blocked. After approval, `AddTaskCommand`s fill the board. `updateTaskStatus()` refuses to mark a task done while a dependency is open. `checkOverdueTasks()` runs at start-up and every morning and alerts the Dashboard.

### F10 — Invitations

**Use cases:** UC10, UC15 · **Sequence diagram:** SD08

**Classes:** `EventController`, `AttendeeController`, `EventPlannerFacade`, `Event`, `EventState` (`PlanningState`, `ConfirmedState`, ...), `InvitationDraftingWorkflow`, `InvitationParser`, `Invitation`, `MessageGateway`, `OutboxFileGateway`.

**Methods:** `transitionEvent()`, `Event.transition()`, `EventState.onAction()`, `Event.setState()`, `draftInvitations()`, `Event.ensureCanInvite()`, `EventState.ensureCanInvite()`, `sendInvitations()`, `MessageGateway.send()`.

**How it works:** When the organizer confirms the event, `Event.transition()` asks the current state (`PlanningState`) to handle it; the state checks its rule and installs `ConfirmedState`. `draftInvitations()` calls `ensureCanInvite()`, which the current state answers: Draft, Completed and Cancelled refuse. The workflow drafts one template per attendee group and checks the facts. After editing, `sendInvitations()` sends each one through `MessageGateway`; failures are listed but do not stop the rest.

### F11 — FAQ Agent (RAG)

**Use case:** UC11 · **Sequence diagram:** SD09

**Classes:** `ChatPanel`, `AgentChatController`, `EventPlannerFacade`, `FaqAnswerWorkflow`, `KnowledgeBase`, `AgentOrchestrator`, `FaqAnswerParser`, `FaqAnswer`, `KnowledgeBaseIndexer`.

**Methods:** `onAskFaq()`, `askFaq()`, `FaqAnswerWorkflow.gatherContext()`, `KnowledgeBase.search()`, `KnowledgeBase.index()`, `AgentOrchestrator.run()`, `FaqAnswerParser.parse()`, `FaqAnswerWorkflow.validate()`.

**How it works:** `KnowledgeBaseIndexer` keeps the knowledge base current by calling `index()` after each change. For a question, `gatherContext()` calls `search()` for the top 5 pieces. If none is relevant, the workflow returns an escalated answer without calling the LLM, and the organizer gets an alert. Otherwise the agent must answer only from the numbered sources, and `validate()` checks that each cited source was really retrieved.

### F12 — Plain-Language Changes with Undo/Redo

**Use cases:** UC12, UC13, UC14 · **Sequence diagram:** SD10

**Classes:** `AgendaCliCommand` / `AgendaController`, `ConsoleProposalReviewer`, `EventPlannerFacade`, `PlanModificationWorkflow`, `CommandMapper`, `CommandHistory`, `MacroCommand`, `MoveSessionCommand`, `AddAgendaItemCommand`.

**Methods:** `AgendaCliCommand.call()`, `onModifyPlan()`, `modifyPlan()`, `CommandMapper.toCommand()`, `ConsoleProposalReviewer.review()`, `applyProposal()`, `CommandHistory.execute()`, `undo()`, `redo()`, `MacroCommand.undo()`.

**How it works:** The prompt contains the agenda with item ids and a fixed list of allowed operations; conversation memory resolves words like "it". If the request is ambiguous, the agent asks a question. Otherwise `CommandMapper.toCommand()` turns each operation into a command, rejecting unknown items. After approval (a dialog in the GUI, `y/N` in the CLI), one `MacroCommand` is executed by `CommandHistory`. `undo()` calls `MacroCommand.undo()`, which reverses the steps in the opposite order, and `redo()` applies them again.

---

*End of Stage 1 report — Mohammed Faraz Kabbo, EECS 3311, Fall 2026.*